Part VII

Micro Robots, Drones and Edge AI SLAM

Simultaneous Localization and Mapping (SLAM)

How to Build a Vision-Based Navigation and Localization Drone



Editor: Lee Shihyuan (李世淵)

LINE OA @prof

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Part VII Micro Robots, Drones, and Edge-AI SLAM

Chapter 25

NanoSLAM and Low-Power SLAM

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

The visual SLAM of Part VI is mature enough, but it rests on a premise it never states aloud: that the robot carries a computer with power to spare, and that it can either be plugged in or carry a battery heavy enough to feed it. This chapter removes that premise and asks what is left.

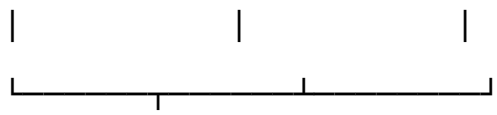
The subject of Part VII is the severely constrained platform—a 27-gram micro drone, a chip that draws 66 milliwatts, a battery that lasts seven minutes. Under that budget the occupancy grid of Chapter 13, the scan registration of Chapter 17, and the key frame management of Chapter 22 must every one of them be weighed again.

The three figures of this chapter are the scales on which they are weighed: Figure 25-1 explains why many cores running slowly use less power than one core running fast; Figure 25-3 asks whether measurements this sparse can still localize and map, and by how much GAP9's energy efficiency really leads; and Figure 25-4 computes something students may find surprising—that the weight of a compute module eats more flight time than its power draw does.

Figure 25-0 The Link Map of Chapter 25

What has been built up so far now goes on the scales

Chapter 8 Ranging sensors Chapter 13 Occupancy grid maps Chapter 17 Scan registration



▼

【Chapter 25 Compressing SLAM into a milliwatt budget】

Sparse ToF ─ the PULP parallel architecture ─ power and weight budgets ─ TinyML

▼

Chapter 26 Drone SLAM Chapter 27 Multi-robot collaboration

▼

Chapter 33 Edge deployment (Jetson, quantization, and distillation)

Learning Objectives

Every system in the first six parts has presumed that the robot has computing resources and electrical power enough—ORB-SLAM3 needs a whole processor, LiDAR SLAM a computing platform of tens of watts. But what if the robot weighs only 44 grams, fits in the palm of a hand, and has a power budget at the milliwatt level? This chapter displays the most astonishing engineering achievement of that field, and opens the extreme challenges of Part VII. After studying this chapter, students should be able to:

1. Explain the three great constraints on running SLAM on a micro robot, and the trouble with the traditional remedy.
2. Explain NanoSLAM's design goals and its five core contributions.
3. Explain how RISC-V and the parallel ultra-low-power computing paradigm underwrite milliwatt-level SLAM.

4. Explain the design trade-offs of a milliwatt-level SLAM system and how power is allocated among its modules.
5. Give examples of applications in micro drones, swarm robots, and AR glasses.
6. Explain the influence of edge AI and TinyML on the development of SLAM.
7. Plan a high-school-level simulation project on mapping with a micro drone.

Core Concepts

中文名詞	English Term	Explanation
微型無人機	Nano-UAV / Nano-drone	An ultra-small aircraft the size of a palm, weighing tens of grams.
機載運算	Onboard Computation	All the computation is done on the robot itself, depending on no external facility.
運算卸載	Computation Offload	Sending the computation to an external computer, subject to the bandwidth and the latency of communication.
平行超低功耗	PULP	A computing paradigm that trades multi-core parallelism for energy efficiency.
系統單晶片	SoC	Integrating processor, memory, and peripherals on a single chip.
開放指令集架構	RISC-V	An open and extensible instruction set architecture for processors.
飛時測距矩陣	ToF Matrix Sensor	A small depth sensor that measures distance in several directions at once.
掃描匹配	Scan Matching	Comparing successive measurements to estimate relative motion (Chapters 13 and 17).
位姿圖優化	Pose Graph	Correcting accumulated error by graph

中文名詞	English Term	Explanation
	Optimization	optimization (Chapter 12).
動態頻率調節	Dynamic Frequency Scaling	Adjusting clock and voltage in real time according to the load, to save power.
深度睡眠	Deep Sleep	A power-saving state that brings the power very low while idle.
邊緣人工智慧	Edge AI	Running AI inference on the device rather than in the cloud.
微型機器學習	TinyML	Running machine learning on severely constrained hardware such as a microcontroller.

25.1 Why SLAM Is So Hard on a Micro Robot

The first question of Part VII is this: why is it so difficult to fit SLAM into a micro robot? The answer is not simply that the chip is slower; it is the superposition of three constraints.

The Three Great Constraints

- Computing power: both the front end of SLAM (feature extraction, scan matching) and its back end (graph optimization, BA) are extremely expensive to compute. The ORB-SLAM family of Chapters 22 and 23 needs a complete multi-core processor; a chip at the microcontroller level simply cannot run it.
- Memory: Chapter 13 did the arithmetic—a 2D map of 20 × 20 meters at 5-centimeter resolution already has 160,000 cells, and the Gmapping of Chapter 14 must multiply that by the number of particles. A micro platform may have only a few hundred kilobytes to a few megabytes.
- Electrical power: this is the most severe constraint of all. A micro drone's battery holds very little energy, and flight itself consumes the overwhelming part of it. Let computation take several watts more and the flight time falls from minutes to tens of seconds—which leaves the system useless.

The Traditional Remedy: Offloading the Computation, and the Trouble with It

Because of the enormous computational load and memory requirement of running a SLAM algorithm, fully onboard mapping can at present be achieved only on robot platforms able to carry a processor of many watts; and precisely for that reason, pocket-sized robots constrained in hardware have had to offload the execution of SLAM to external facilities.

This practice carries obvious costs:

- Dependence on infrastructure: a wireless network and an external computer are required, and the robot is never truly autonomous. At a disaster site, in an underground tunnel, or out in the field this is simply not workable.
- Communication latency: the images or sensor data go out and the commands come back, and the delay may reach tens to hundreds of milliseconds. For a drone flying fast, that delay may mean flying into a wall.
- Bandwidth and power: continuously transmitting large volumes of data itself consumes power, and when several robots operate at once bandwidth becomes the bottleneck (the multi-robot question of Chapter 27).
- Reliability: let the network drop and the robot loses its ability to navigate.

The Central Question of This Chapter

The question this chapter must answer is therefore: can SLAM be made to run entirely onboard, at a power low enough for a micro drone to afford?

The answer is yes, and it has already been done. It required a simultaneous breakthrough at three levels: the algorithm had to be made light, the hardware had to be made efficient, and the sensor had to be made frugal. The sections that follow are the story of how those three were made to work together.

25.2 The Design Goals of NanoSLAM

NanoSLAM was proposed in 2023 by teams at ETH Zürich and the University of Bologna, and is the representative work referred to repeatedly in Chapters 7 and 25. Its goal is clear and bold: to let a micro drone run SLAM entirely by itself.

The Central Claim

NanoSLAM is a lightweight and optimized end-to-end SLAM method designed to run on centimeter-sized robots within a power budget of only 87.9 milliwatts. The research team demonstrated its mapping ability in real scenes and deployed NanoSLAM on a micro drone weighing 44 grams and carrying GAP9, a new commercial RISC-V low-power parallel processor.

To give students a feel for these numbers: 87.9 milliwatts is about the power of an LED indicator lamp, and 44 grams is lighter than an egg. To run complete SLAM under such conditions—this is the most astonishing thing in the chapter.

Five Core Contributions

The NanoSLAM paper lists five contributions explicitly, and they form the technical skeleton of this chapter:

1. A parallel optimized implementation: an optimized parallel implementation of the algorithms, with an assessment of their execution time and memory requirement.
2. Onboard ICP: a parallel implementation and evaluation of ICP, the benchmark algorithm of scan matching (Chapters 13 and 17), able to complete onboard in 55 milliseconds.
3. A sensing expansion board: a custom add-on companion board for the commercial Crazyflie 2.1 micro drone, extending the drone's sensing with four ToF matrix sensors so that it can perform scan matching and autonomous navigation.
4. A communication protocol: a protocol coordinating the integration and the exchange of data between the drone's original microcontroller and the GAP9 system on chip, prescribing how the graph structure is stored, how the poses of

the graph are exchanged, how edges are added, and how graph optimization is performed.

5. Validation in the field: an extensive field-experimental evaluation demonstrating the system's loop-closing mapping ability—a reduction in trajectory error of up to 67 percent with NanoSLAM, and a mapping accuracy of 4.5 centimeters.

The Key Result Figures

The design of this algorithm makes good use of the parallel capability of the RISC-V processing cores, and can build a map of an ordinary environment to an accuracy of 4.5 centimeters with an end-to-end execution time under 250 milliseconds.

Set these numbers beside the earlier chapters and their value is felt more keenly: the 3.5-centimeter accuracy of ORB-SLAM3 on EuRoC in Chapter 23 was obtained on a complete computer, while NanoSLAM's 4.5 centimeters was achieved on a milliwatt-level micro platform. The research team points out that by combining a new kind of low-power depth sensor with GAP9's parallel capability, the system attains an indoor mapping accuracy comparable to that of larger and computationally more powerful drones (usually called micro air vehicles or standard-sized drones).

25.3 RISC-V, GAP9, and Low-Power Parallel Computing

However light the algorithm, it needs suitable hardware to be realized. This section introduces the key hardware that makes milliwatt-level SLAM possible.

New Trends in Microcontroller Design

Recent trends in microcontroller design stress parallel processing, hardware accelerators, and energy efficiency. GreenWaves Technologies' GAP9 system on chip is

exactly the embodiment of these trends, suited to the specialized application of a micro drone; with RISC-V cores it supplies advanced parallel capability, power optimization, and sensor integration, giving a micro drone real-time edge computing, longer flight time, and stronger data processing.

PULP: the Parallel Ultra-Low-Power Paradigm

GAP9 is built on the parallel ultra-low-power (PULP) computing paradigm; it is small, and draws under 180 milliwatts.

The central idea of PULP is worth understanding, because it runs against intuition: rather than one core that runs very fast, use several cores that run slowly and work in parallel. The reason is that the power of a chip is roughly proportional to the square of the voltage and to the frequency—halve the frequency and let two cores work in parallel, and the total amount of computation is the same while the power may fall to less than half. This is the engineering wisdom of trading parallelism for efficiency.

A Comparison of Chip Specifications

Setting side by side the several chips common on micro drones shows plainly where GAP9 stands:

Chip (architecture)	CPU cores	Max frequency	Power	Where it sits
STM32 (ARM)	1	168 MHz	116 mW	Flight control and state estimation
ESP32 (Xtensa)	2	240 MHz	300 mW	Communication and light computation
GAP8 (RISC-V)	8 + 1	175 MHz	96 mW	The previous generation of AI acceleration

Chip (architecture)	CPU cores	Max frequency	Power	Where it sits
GAP9 (RISC-V)	9 + 1	370 MHz	66 mW	The subject of this chapter

Table 25-1 A Comparison of Systems on Chip Common on Micro Drones

This table says three things. First, GAP9 has far more cores than a traditional microcontroller, which embodies the parallel thinking of PULP. Second, even with more cores and a higher frequency its power is lower, so its advantage in efficiency is plain. Third, the STM32 used for flight control and the GAP9 used for computation stand in a division of labor—which is exactly why the fourth contribution of Section 25.2, the communication protocol, exists at all.

Why RISC-V

RISC-V is an open instruction set architecture that anyone may use and extend freely, with no licensing fee. This is extremely favorable to research and to startups:

- It can be customized: special instructions can be added for a particular computation (the matrix arithmetic of SLAM, say), raising efficiency greatly.
- There is no licensing cost: for academic research and small production runs the effect on the cost structure is appreciable.
- The ecosystem is growing fast: RISC-V has risen quickly in edge AI and the internet of things in recent years, and its tool chains are maturing.

For the reader of this book the value of knowing RISC-V is this: it represents the trend by which computing hardware is no longer monopolized by a few manufacturers, and it is a direction worth watching for Taiwan's semiconductor and robotics industries.

Advanced Features: Dynamic Scaling and Deep Sleep

GAP9 supports adjusting the clock frequency (up to 370 MHz) and the voltage at run time, together with automatic clock gating; its idle power can fall to 45 microwatts.

This means the system can run fast where it must and save where it may: at full speed while running SLAM, and at a reduced frequency or even in deep sleep while flight is steady and no mapping is needed. An idle power of 45 microwatts is all but negligible—and this fine-grained power management is exactly the key to a longer flight time.

Measuring It: Do Many Cores Running Slowly Really Use Less Power

Section 25.3 says that rather than one core that runs very fast one should use several cores that run slowly in parallel, on the ground that power is roughly proportional to the square of the voltage and to the frequency. The inference is worth working through with real arithmetic, because it has an equally important second half: more cores are not always better.

The model is simple. To reach a fixed total throughput, N cores each need run at only 1 / N of the frequency. The dynamic power is $N \times C \times V^2 \times f$, and in the near-threshold region the voltage falls roughly linearly with the frequency, but with a floor (a transistor cannot go below the threshold voltage). On top of that every core contributes leakage current, which has nothing to do with the frequency.

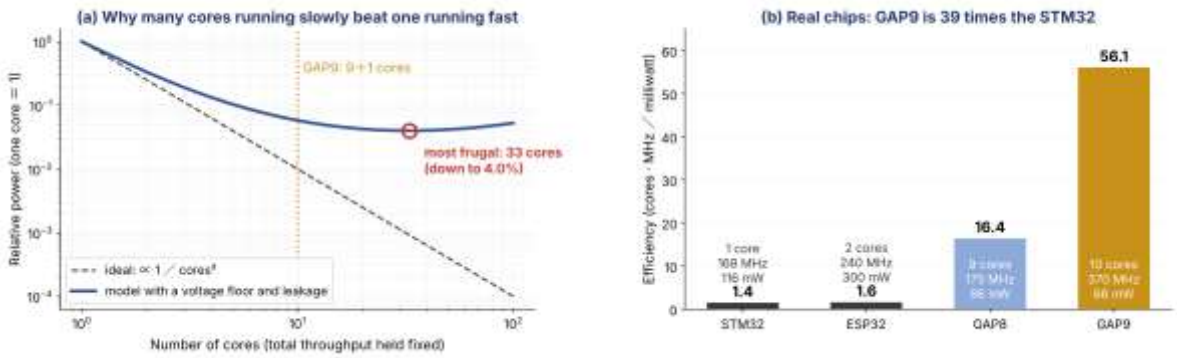


Figure 25-1 The Energy-Efficiency Principle of PULP Set Against Real Chips

Panel (a) is the result of the model, with both axes logarithmic. The gray dashed line is the ideal case—if the voltage could fall with the frequency without limit, the power would be proportional to the inverse square of the number of cores, which is to say that doubling the cores would quarter the power. The blue line is the real curve once the voltage floor and the leakage are included: it hugs the ideal line at first, the power falling to 6.8 percent from 1 core to 8, then departs from it gradually, bottoming out at about 32 cores (down to 4.0 percent), and rising again beyond that because of leakage.

That minimum explains something students may not have thought to ask: if many cores save power, why does GAP9 carry 9 + 1 cores and not ninety? Because beyond the optimum, the leakage each additional core brings exceeds the dynamic power it saves. The optimal number of cores on a real chip must further take account of the cost in area, of memory bandwidth, and of parallelization efficiency—a real workload can seldom be cut into ninety perfect pieces.

Panel (b) verifies the principle against the real chip figures of Table 25-1. Taking the number of cores times the frequency as a rough index of throughput and dividing by the power, the efficiencies are: 1.4 per milliwatt for STM32, 1.6 for ESP32, 16.4 for GAP8, and 56.1 for GAP9. GAP9 is 39 times STM32 and 35 times ESP32.

That multiple does not come entirely from parallelism. From GAP8 to GAP9 the efficiency leaps from 16.4 to 56.1 (3.4 times) while the number of cores rises only from 9 to 10—the difference comes mainly from improvements in process, in the memory hierarchy, and in dedicated accelerators. In other words, the parallel thinking of PULP supplies the first order of magnitude and the hardware optimizations that follow supply another. Only with the two superposed does SLAM come down from watts to milliwatts.

25.4 The System Design of Milliwatt-Level SLAM

This section explains how NanoSLAM compresses the technology of Chapters 13 and 17 into a milliwatt budget.

The Choice of Sensor: a ToF Matrix Rather Than a LiDAR or a Camera

The rotating 2D LiDAR introduced in Chapter 8, useful as it is, is too heavy and too power-hungry for a micro drone. A camera needs a great deal of computation before depth can be extracted (Chapter 9). NanoSLAM's choice is the ToF matrix sensor—a small time-of-flight ranging chip that measures the distance in several directions at once.

- Its merits: a few millimeters in size, extremely low power, a distance output directly (with no computation needed), and no moving parts.
- Its arrangement: as described in Section 25.2, four ToF matrix sensors face in four directions and together supply a distance measurement over approximately 360 degrees—which amounts to a minimal version of a 2D LiDAR.

- Its limitations: the resolution and the measuring range are far below those of a true LiDAR, so the algorithm must be able to work on sparse measurements.

The Algorithmic Pipeline

The overall flow of NanoSLAM is an architecture students already know well; it is only that every module has been made light to the last degree:

Figure 25-2 The System Pipeline of NanoSLAM

【Sensing】4 ToF matrix sensors → sparse distance measurements over approximately 360 degrees

↓

【Front end】Parallelized ICP scan matching (Chapters 13 and 17)

estimates the relative pose between successive instants → completed onboard in 55 ms

↓

【Building the graph】poses stored as nodes, the results of scan matching as edges (the pose graph of Chapter 12)

↓

【Loop closure detection】recognize a revisited position → add a loop closure edge

↓

【Back end】Parallelized pose graph optimization (Chapter 12)

corrects the accumulated drift → trajectory error reduced by up to 67%

↓

【Output】a map and a trajectory completed onboard end to end < 250 ms, at 87.9 milliwatts

Three Key Strategies for Keeping It Light

1. Parallelize rather than simplify: NanoSLAM did not turn ICP or graph optimization into a simple approximate version; it parallelized the standard

algorithms across many cores. This preserves the accuracy while making good use of what the PULP hardware is good at.

2. Sparse rather than dense: sparse measurements from a ToF matrix take the place of a dense point cloud, which greatly reduces both the volume of data and the computation—of a piece with LOAM's idea in Chapter 18 that not all the points are needed.
3. Divide the labor rather than do everything: flight control and state estimation are the business of the original STM32 and the SLAM computation that of GAP9, the two coordinated by a dedicated communication protocol. Each to its own office, and no single chip overloaded.

What the Allocation of Power Reveals

Putting the relevant figures together shows the trade-offs of the system design:

Task	Power / latency	Note
NanoSLAM mapping	about 87.9 mW, latency < 250 ms	The subject of this chapter
Monte Carlo localization	23 mW on average	Localization given a known map (Chapter 11)
YOLO object detection	below 100 mW, as little as 17 ms	Edge AI inference (Chapters 24 and 32)
Deep sleep	45 µW	Very low power while idle

Table 25-2 The Power Performance of Each Task on a Micro Platform

Holding the system within a power envelope below 100 milliwatts—above all for computationally intensive tasks—is vital to extending flight time and to supporting complex real-time task loads.

It is worth noting that localization (23 mW) uses far less power than mapping (87.9 mW). This bears out the observation of Case 14-3 in Chapter 14—that mapping for the first time and localizing afterwards are different tasks. For a micro robot it implies a practical strategy: build the map well first (it may be slower and more costly in power), and let the tasks that follow need only low-power localization.

Measuring It: With Measurements This Sparse, Can It Still Localize

Section 25.4 says that the resolution and measuring range of a ToF matrix are far below those of a true LiDAR, so that the algorithm must be able to work on sparse measurements. That sentence leaves two questions unanswered: how sparse may it be and still be usable? And if something must be given up, should one keep the number of rays or the range? This subsection measures the two separately.

The experimental setting is an indoor scene of 10 × 7 meters containing six pieces of furniture. The robot moves along an elliptical trajectory, and at each position estimates its own pose by likelihood-field scan registration (the method of Chapter 17) and compares it with the true pose. Two sensor configurations are used: four ToF matrices (45 degrees of field of view each, about 180 degrees in all) and a rotating LiDAR (360 degrees, 12 meters).

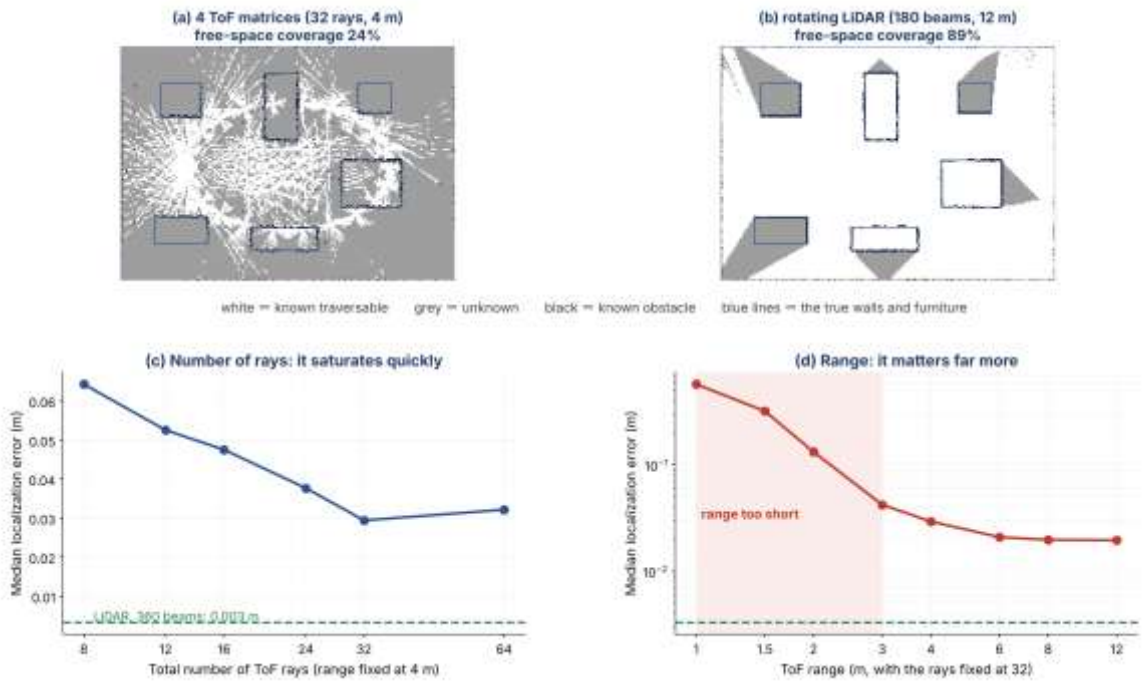


Figure 25-3 Mapping and Localization with a Sparse ToF and with a Rotating LiDAR

Panels (a) and (b) are the maps the two build. The LiDAR sweeps the room clean, with 89 percent coverage of the free space and the outlines of all six pieces of furniture clear. The ToF map is plainly incomplete: the coverage is only 24 percent, the white traversable region radiates outward in spokes, and large areas remain the gray of the unknown. There are two reasons—four sensors cover only about half of the bearings, and the range is only 4 meters, which cannot reach the far walls.

Panel (c) answers the first question, that of the number of rays. With only 8 the localization error is 0.064 meters; raise it to 12 and it falls to 0.053, to 16 and it falls to 0.048, and by 32 rays it has reached 0.030 meters, after which the curve is flat—0.032 at 64. That is to say, going from 32 rays to 64 (twice as many) buys nothing at all. This is a conclusion of real importance: for localization, a few dozen measurements already come close to saturation.

Panel (d) answers the second question, that of range, and the answer is entirely

different. At a range of 1 meter the localization error is 0.576 meters (not usable at all), at 2 meters 0.133, at 4 meters 0.030, and only beyond 6 meters does it flatten at 0.021 meters. From 1 meter to 4 the error improves nineteenfold. Set panels (c) and (d) side by side and the conclusion is plain: in choosing a ToF sensor for a micro platform, range matters far more than the number of zones.

Finally, set the ToF beside the LiDAR: a 4 × 8 ToF arrangement at a range of 4 meters gives a localization error of 0.030 meters against the LiDAR's 0.003—nine times worse, with an eleventh of the rays and a third of the range. This is exactly what makes milliwatt-level SLAM possible: localization needs far less density of measurement than intuition suggests, though it does need to see far enough.

Enough for Localization Does Not Mean Enough for a Map

There is a gap in this figure that must be made clear: the ToF's localization accuracy is nine times worse than the LiDAR's, but its map coverage is barely a quarter of the LiDAR's (24 percent against 89).

The reason is that the two need different quantities of measurement. Localization fits the current frame to a known structure, and a few correct distance measurements are enough to pin down six degrees of freedom; mapping, on the other hand, must actually sweep every piece of space once, and there is no shortcut.

This also explains why a real milliwatt-level system mostly builds only a coarse map that will serve—not because the algorithm is wanting but because the sensor's total quantity of measurement is simply that much. In planning a project on a micro platform, students should set the goal at being able to return to the start and avoid obstacles rather than at building a map as

handsome as the LiDAR's.

25.5 Applications: Micro Drones, Swarm Robots, and AR Glasses

What applications, impossible before, does milliwatt-level SLAM open up? This section describes three representative directions.

Micro Drones: Safety and Accessibility

A micro drone fits in the palm of the hand, suits indoor environments, and is very safe around people; yet such drones often cannot deliver the performance a complex task requires, for want of sufficient sensing and computing hardware.

Safety deserves particular emphasis. A standard drone of several kilograms that goes out of control and strikes a person may cause serious injury; a 44-gram micro drone is next to harmless. This lets them operate in classrooms, offices, hospitals, and other spaces where people are—which a large drone cannot do.

- Indoor exploration and survey: exploring and mapping spaces people cannot or should not enter (narrow ducts, buildings after a disaster, ceiling voids).
- Agricultural inspection: research already exists in which several autonomous micro drones detect pests visually and a single slower heavy vehicle then goes to the detected outbreak to apply treatment; to meet the extreme constraints of a micro drone (a low-resolution sensor and a computing power budget below 100 milliwatts), the team designed and optimized a tiny image-based convolutional neural network for pest detection.

- Education and research: a micro drone is relatively affordable and very safe, and makes an excellent teaching platform (Section 25.7).

Swarm Robots

Once both the cost and the size of a single robot have come down, using a great many of them becomes a workable strategy. The value of swarm robots lies in:

- Efficiency of coverage: several robots explore separately and the time to finish a large task falls greatly (the multi-robot collaborative exploration of Case 24-3 in Chapter 24).
- Fault tolerance: the failure of one does not affect the task as a whole—an advantage a single expensive robot cannot match.
- Distributed perception: observing from several viewpoints at once allows a more complete model of the environment to be built.

But swarm robots have one crucial premise: every robot must be able to localize itself. If all of them depend on external facilities or a central computer, the communication bandwidth collapses as the number of robots grows (as Section 25.1 noted). Onboard SLAM is therefore a necessary condition for swarm robots to scale—and that is exactly the strategic value of a technology like NanoSLAM. Chapter 27 goes deeply into the collaborative questions of multi-robot SLAM.

AR Glasses and Wearable Devices

Chapters 20 and 23 discussed AR's need for visual-inertial SLAM, but the constraints on AR glasses are more severe than those on a phone: they must be light enough to wear for long periods, the battery must be small, and they must not grow hot against the face.

These constraints are astonishingly like those on a micro drone—both amount to running SLAM within a very small power budget. The technical line of this chapter (a dedicated low-power chip, parallel ultra-low-power computing, a lightweight algorithm) therefore applies directly to wearable devices too.

This commonality across fields is an important idea in this chapter: low-power SLAM is not a specialized technology for micro drones alone, but a shared requirement of every constrained platform. Master this line and students hold the key to a great many applications—AR glasses, smart watches, miniature sensing nodes, and more.

Measuring It: Is the Price of Computation Really Its Power

Section 25.1 says the difficulty for a micro robot lies in the three constraints of computation, power, and weight. Which of the three is the most deadly? Intuition says power, but the arithmetic makes the answer weight—and by a wide margin.

The basis of the calculation is a micro drone of the 27-gram class: a battery of about 0.89 watt-hours, a hover power of about 7.6 watts, and an endurance in pure hover of about 7.0 minutes. By momentum theory the hover power is roughly proportional to the mass raised to the power 1.5, so hanging one more board on it does not merely consume that board's own power—it makes all four motors work harder.

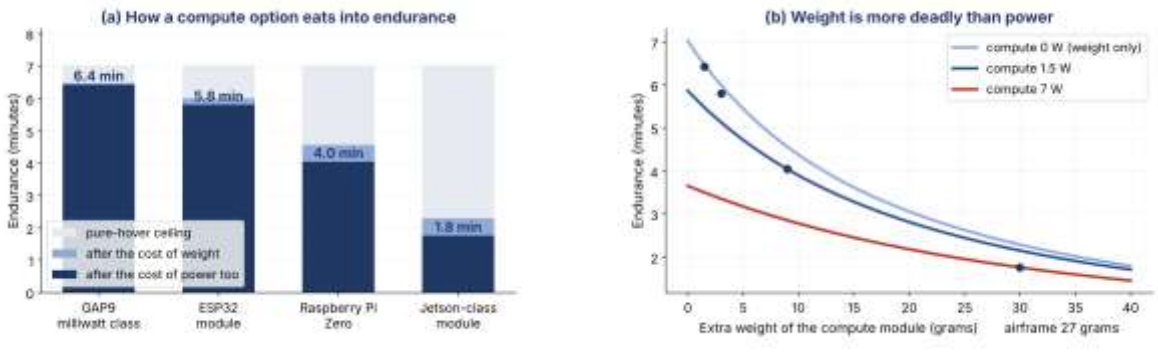


Figure 25-4 How the Weight and the Power of a Compute Option Eat into

Endurance

Panel (a) splits the endurance loss of four compute options into two parts. Take GAP9: at only 1.5 grams and 66 milliwatts, its weight brings the endurance down from 7.03 minutes to 6.48, and its power brings it down further to 6.43—a loss of only 8.5 percent in all.

A module of the Jetson class is the other extreme: 30 grams and 7 watts. Its weight presses the endurance from 7.03 minutes down to 2.29, and its power presses it further to 1.76. Note the ratio of these two parts—the weight costs 4.74 minutes and the power only 0.53, the former nine times the latter. For the four options this ratio is 10.6, 5.3, 4.7, and 9.0 times, without a single exception.

Panel (b) takes the weight out on its own. Even with a compute power of 0 watts (imagine a board that draws nothing at all), simply hanging 30 grams on the drone brings the endurance down from 7.0 minutes to 2.3. The vertical distance between the three curves—which is to say the effect of power—is far smaller than the fall of the curves themselves.

This conclusion bears directly on the choice of parts for a project. In choosing a compute module for a micro platform the first question to ask is not how many watts it draws but how much it weighs. A frugal but bulky board may cost more endurance than a thirsty but light one. This is also why Section 25.2 insists that milliwatts and a few grams must hold together—let either fail and micro SLAM does not stand.

When Does This Rule Reverse

The conclusion that weight comes first presumes that the hover power far exceeds the computing power. For a 27-gram drone, 7.6 watts against 0.066 is

a difference of more than a hundredfold, so weight dominates entirely.

Change to a ground robot and it is otherwise. A wheeled robot of 5 kilograms may use only 10 watts to move, and hanging another 100 grams on it scarcely affects the energy at all—there the computing power becomes the main consideration instead.

So this is not a universal rule but a rule for flying platforms. Before applying it, students should compute for their own platform the ratio of the power of motion to the power of computation, and the ratio of the added weight to the platform's own.

25.6 The Influence of Edge AI and TinyML on SLAM

This section widens the view from SLAM to the trend of edge intelligence as a whole, and explains its links with the later chapters of this book.

What Edge AI Means

Edge AI means running AI inference on the device rather than in the cloud. Its value is exactly of a piece with the reasons Section 25.1 gave for onboard SLAM: no dependence on a network, low latency, protection of privacy, and the ability to scale.

TinyML is the extreme version of edge AI—running machine learning on hardware such as a microcontroller with only a few hundred kilobytes of memory. It sounds impossible, but through model quantization, pruning, knowledge distillation, and the like, useful inference can already be run on very small hardware.

Three Ways It Joins with SLAM

1. Enhanced perception: running object detection on a micro platform. Benchmarks show that an inference task such as YOLO object detection can be completed below 100 milliwatts, with a shortest inference time of 17 milliseconds at about 1.59 millijoules. This lets a micro robot do semantic perception too (Chapter 24).
2. Learning in place of computation: using a small neural network to estimate certain quantities directly (optical flow, depth, even pose) in place of traditional geometric computation. If the model is small enough, inference may use less power than the traditional algorithm.
3. Online learning: a more advanced direction still is to let the robot adapt by learning for itself while in flight. Research already exists on on-device self-supervised learning aboard a micro drone within an extremely low power budget—which means the robot does not merely execute a model trained in advance but can keep adapting to new environments.

The Co-Design of Hardware and Algorithm

The most important methodological lesson of this chapter is this: under extreme constraints, hardware and algorithm cannot be designed apart.

NanoSLAM succeeded not because the algorithm was especially clever, nor because the chip was especially fast, but because the two were fitted to each other: the algorithm was tailored to the parallel architecture of the RISC-V multi-core, and the hardware supplied the efficiency and the integration the algorithm required. This way of thinking—hardware-software co-design—is a core competence in the field of edge computing.

For students of an intelligent robotics program this is an important reminder:

besides learning to use algorithms off the shelf, understanding the properties of the underlying hardware (parallelism, the memory hierarchy, the behavior of power) matters just as much. Where those two meet is exactly where Taiwan's advantage in semiconductors and its applications in robotics can be joined.

Links with the Chapters to Come

- Chapter 26: visual-inertial navigation for a micro drone, continuing this chapter's discussion of the constraints of the platform.
- Chapter 27: multiple robots and cloud-edge collaboration, answering to the swarm robots of Section 25.5.
- Chapters 32–34: semantic SLAM, training in simulation, and edge-AI deployment (Jetson, the NPU; Chapter 33), together with VLM language navigation (Chapter 34), are the extension of this chapter's discussion of edge computing.

25.7 A High School Project: Simulating Mapping with a Micro Drone

The technology of this chapter sounds out of reach, but it in fact makes an excellent high school project—because the central ideas can be experienced in full through simulation, with no expensive hardware.

Why It Suits High School

- The idea is intuitive: building a map with distance sensors in four directions is far easier to understand than the features and the geometry of visual SLAM.
- It can be simulated completely: a 2D simulation of a micro drone can be written in Python, with no aircraft needed.

- The connection is felt: micro drones, swarm robots, and AR glasses are all subjects students find interesting.
- The levels are distinct: from the simplest sensing and drawing, to adding drift, to correcting by loop closure, the difficulty can be dialed up.

The Design of the Project: Four Stages

1. Stage one—build the simulated environment: draw a 2D room (walls and obstacles) in Python, put a virtual micro drone in it, and implement distance sensing in four directions (from the drone's position, find the nearest wall in each of four directions).
2. Stage two—ideal mapping: fly the drone along a prescribed route and at every step use the four distance measurements to draw the detected wall points onto the map. This stage assumes the position is perfectly accurate, and students will watch a handsome map of the room emerge bit by bit.
3. Stage three—add drift: put a small random error into the drone's estimate of its own position (simulating odometry drift, Chapters 1 and 7) and build the map again. Students will see with their own eyes the map begin to distort and the wall lines double—a concrete reproduction of the map pathologies of Section 13.7.
4. Stage four—correct by loop closure: let the drone go once around and return to the start, add the constraint that the end should equal the start, and spread the error back over the whole trajectory by a simplified method (the paper method of Implementation 12-1 in Chapter 12 applies directly), then watch the map being pulled straight.

Directions for Extension

- Computing the power budget: given the battery capacity and the power of each module (see Table 25-2), compute the flight time, and experiment with how long it can fly if the mapping rate is halved—an excellent exercise in engineering trade-offs.
- Swarm simulation: add several virtual drones exploring separately and compare the time needed to cover the whole area (answering to Section 25.5 and Chapter 27).
- An experiment on the number of sensors: compare distance sensing in 4, 8, and 16 directions and its effect on the quality of the map—which corresponds to the trade-off in a real system between the number of sensors and the power and weight.

Teaching Suggestions

The teaching value of this project is that it lets students walk through the whole core logic of SLAM in the most simplified setting: sense → map → discover the drift → correct by loop closure. This is a miniature of Parts I to IV of this book.

Teachers are advised to let students first produce the perfect map of stage two and feel the achievement, and then to add drift deliberately to break it—that moment when the handsome map suddenly goes crooked will make students truly understand why SLAM is a hard problem. This rhythm of building, then breaking, then repairing is far more effective than expounding the theory directly.

At this point students have seen SLAM realized on a platform under extreme constraint. The next chapter focuses on the navigation challenges of the drone itself: how is one to fly in a GPS-denied environment? How are fast motion and vibration to be handled? How does visual-inertial odometry work on a drone? Chapter 26 carries

the discussion of Part VII to the heart of flight.

25.8 After the Milliwatt: Three Advances of 2025–2026

NanoSLAM proved that milliwatt-level SLAM is possible. The advances of the last few years have been mainly in three directions, and the price of each is worth knowing.

One: Moving a Complete Visual-Inertial Odometry onto RISC-V

A piece of work from 2026 deployed a complete monocular visual-inertial pipeline onto GAP9: ORB features, the eight-point algorithm and EPnP with RANSAC, IMU preintegration, and sliding-window pose graph optimization—which is to say the whole apparatus of Chapters 21 and 23, packed into a chip running at 370 MHz.

The result is 20 frames per second, a power below 100 milliwatts, and a memory footprint under 1 MB (about 110 KB in the first level and about 850 KB in the second). But the price is explicit: it does without loop closure detection, and on the standard EuRoC dataset its mean trajectory error is about 3.46 meters against about 0.25 meters for ORB-SLAM3 with loop closure—a difference of an order of magnitude. The authors themselves attribute the reason to the absence of global correction.

This comparison is worth reading together with Figure 22-2 in Chapter 22. That figure proved that loop closures and covisibility edges are the only route by which error is redistributed; and a micro platform gives them up not because they do not matter but because they require the whole map and the whole trajectory to be kept—which is exactly what will not fit in 1 MB of memory.

Two: Sensors Grow Sparser and Cheaper

The second direction is to simplify the sensor further still. The conclusion of Figure 25-3(c)—that a dozen or so measurements already come close to saturation—is precisely

the theoretical basis of this direction. Practices that have appeared in recent years include replacing several fixed sensors with a single multi-zone ToF on a rotating mount, and using a grayscale camera of very low resolution (160 × 160, say) for optical flow and place recognition.

The common logic of such designs is this: rather than pursue the quality of a single measurement, increase the number and the variety of measurements and then fuse them by the probabilistic methods of Chapter 11. It is the same belief as that of Chapters 8 and 9—that there is no all-powerful sensor, only a fitting combination—only pushed to its limit under a milliwatt budget.

Three: Computation Moves Toward Computing Only When It Must

The third direction lies neither in the algorithm nor in the hardware but in scheduling. Section 25.3 noted that GAP9's idle power can fall to 45 microwatts, which makes event-driven computation worth its while: reduce the frequency or even sleep while flight is steady, and run at full speed only when turning, when meeting an obstacle, or when the uncertainty of localization is detected to be rising.

This idea is of a piece with the map maintenance of Section 19.6 and the staggered execution of Section 33.8—all of them ask whether this step really needs doing now. For a platform with only seven minutes of endurance, saving half the computing time amounts to nearly half a minute more in the air.

	The NanoSLAM of This Chapter	A complete VIO on RISC-V	Event-driven scheduling
Sensor	4 ToF matrices	A monocular camera + an IMU	Not restricted

Loop closure	Yes (lightweight)	No	Depends on the configuration
Memory	A few hundred KB	Under 1 MB	Unchanged
Power	Milliwatt level	Below 100 milliwatts	The average can fall further
Accuracy	Centimeter level (a small site)	About 3.46 meters on EuRoC	Unchanged
The chief price	A sparse map	No global correction	Complexity of implementation

Table 25-3 A Comparison of the Three Directions

Formulas and Procedures

One: The Relation Between Power and Flight Time

The flight time of a micro drone is decided by the battery energy and the total power:

$$\textit{flight time} = \textit{battery energy} / (\textit{flight power} + \textit{computing power} + \textit{sensing power})$$

Taking the Crazyflie 2.1 as an example, a single flight with a 380 mAh battery reaches about 7 minutes. Supposing that the power of flight itself is far greater than that of computation, then:

$$\textit{if computing power is a fraction } r \textit{ of the total power, the flight time falls by about } r$$

This simple relation explains why milliwatts matter so much. If SLAM drew 5 watts, then for a micro drone of about 10 watts in total the flight time would be cut straight

in half; whereas 87.9 milliwatts is only about 1 percent and scarcely affects endurance at all. This is the engineering significance of NanoSLAM's pressing the power down to the milliwatt level.

Two: The Upper Bound on the Benefit of Parallel Speedup

When an algorithm is parallelized across N cores, the theoretical speedup is limited by the part that cannot be parallelized:

$$speedup\ S(N) = 1 \ / \ ((1 - p) + p \ / \ N)$$

where p is the fraction that can be parallelized. This is called Amdahl's law. With p = 0.9 and N = 8, for instance, the speedup is about 4.7 times rather than 8.

This formula bears directly on the design of NanoSLAM: the nearest-point search of ICP is highly parallel (every point searches independently), so p is high and the benefit of parallelism marked; whereas the sparse matrix factorization within graph optimization is harder to parallelize completely. Understanding which parts are worth parallelizing is the key to making good use of the PULP architecture.

Three: The Measure of Energy Efficiency

In comparing the computing power of different platforms, speed alone is not enough; what matters is how much computation can be done per joule:

$$energy\ efficiency = amount\ of\ computation \ / \ energy\ consumed \quad (common\ units: GOPS / W\ or\ Ops / J)$$

This is precisely the goal the PULP paradigm pursues (Section 25.3). Taking this chapter's data as an example, YOLO object detection has a shortest inference time of 17 milliseconds at about 1.59 millijoules—and measuring by how much energy each inference consumes reflects the real usability of a constrained platform far better than

how many inferences it can run per second.

Four: A Decision Procedure for the Design of Low-Power SLAM

Figure 25-5 A Decision Procedure for SLAM Design on a Constrained Platform

① What is the power budget? (battery capacity ÷ target running time − the power needed to move)

↓

② The choice of sensor: what can be afforded within the budget?

• a few watts → 3D LiDAR / depth camera

• a few hundred milliwatts → 2D LiDAR / monocular camera

• a few tens of milliwatts → ToF matrix sensor (this chapter's solution)

↓

③ The choice of algorithm: can the volume of data and of computation be afforded?

• a dense method → needs much memory and computation; not feasible on a constrained platform

• a sparse / feature-based method → a great reduction (the shared idea of Chapters 18 and 20)

↓

④ Assess the parallelism: which parts can be parallelized? (estimate the benefit by Amdahl's law)

↓

⑤ Power management: dynamic frequency scaling, deep sleep, switching mode by task

↓

⑥ Design the division of labor: are flight control and computation to be separated onto different chips? How is the communication coordinated?

Cases

Case 25-1 The Power of One LED: Rethinking the Cost of Computation

What is most worth stopping to think about in this chapter is that figure of 87.9 milliwatts.

Let us make a few comparisons. A phone's LED flashlight consumes some hundreds of milliwatts; a laptop running ORB-SLAM3 may draw tens of watts in the processor—some hundreds of times NanoSLAM. And within such a budget NanoSLAM attains a mapping accuracy of 4.5 centimeters, comparable with that of larger and computationally more powerful drones.

This comparison overturns a common intuition: we tend to think that a better result requires more computation. But NanoSLAM proves that through the co-design of algorithm and hardware, comparable accuracy can be maintained while the power falls by two or three orders of magnitude.

This gives students two lessons. First, the bottleneck in performance often lies not in a want of computing power but in the use of the wrong method. LOAM's not all the points are needed in Chapter 18, ORB's trading some accuracy for speed in Chapter 20, and this chapter's trading parallelism for efficiency are all the same way of thinking: find what is essential to the problem and compute only what is necessary.

Second, innovating under constrained resources often calls forth more real

engineering wisdom than working with resources to spare. This has particular meaning for Taiwan's robotics and semiconductor industries: we may not be able to compete with the giants on raw computing power, but on the road of doing more with less there is ample room to work.

Case 25-2 Who Flies and Who Thinks: the Philosophy of a Two-Chip Division of Labor

One design in the NanoSLAM system that is easily overlooked and yet full of engineering wisdom is its two-chip architecture: the original STM32 handles flight control and state estimation, the added GAP9 handles the SLAM computation, and the two are coordinated by a specially designed communication protocol—covering how the graph structure is stored, how the poses of the graph are exchanged, how edges are added, and how graph optimization is performed.

Why not have one chip do everything? There are three reasons:

- The demands on timeliness differ: the flight control loop must run at a very high rate with very low latency, and any delay may bring the aircraft down; SLAM may be slower (250 milliseconds is acceptable). Put the two on the same chip and a peak in the SLAM computation may drag the flight control down with it.
- Isolation for reliability: even if SLAM crashes, flight control can keep the aircraft safe—a basic principle of safety design.
- Each to its strength: the STM32 suits real-time control and GAP9 suits parallel computation. Use the right chip for the right thing and the efficiency of the whole is best.

Students should notice that this division into a fast reflex layer and a slow

deliberative layer is exactly the same design pattern as the hybrid architecture of Chapter 3, the local and global subsystems of Cartographer in Chapter 15, and the high-rate odometry and low-rate mapping of LOAM in Chapter 18.

This shows one thing: a good principle of system design holds generally across levels. Whether at the level of the algorithm, of the software architecture, or of the hardware, layering by time scale and giving each layer its own office is an effective strategy. Mastering this way of thinking in patterns is worth more than remembering the details of any single system.

Case 25-3 A Micro Drone in the Field: from the Laboratory to the Farm

The value of a micro drone is displayed to the full in one concrete agricultural application.

A research team proposed the following idea: efficient crop production requires that an outbreak of pests be detected early and treated in time, and they considered using a fleet of several autonomous micro drones to detect pests visually, after which a single slower heavy vehicle goes to the detected outbreak to apply treatment.

This division of labor between light cavalry for reconnaissance and heavy troops for the work is extremely clever: micro drones are cheap, safe, and deployable in quantity, and so suit a fast search over a wide area; the heavy vehicle, slow as it is, can carry the chemical and do the actual work. Together they are far more efficient than either alone.

The technical challenge is equally worth noting: to meet the extreme constraints of a micro drone (a low-resolution sensor and a computing power budget below 100 milliwatts), the team designed, tuned, and optimized a tiny image-based convolutional

neural network for pest detection. This network needs only 0.58 GOps per inference, and on their dataset it reaches a mean average precision of 0.79 in detecting harmful insects—14 percent lower in mean precision than the best-performing network in the literature, but with 32 times less computation.

This trade of 14 percent of accuracy for 32 times less computation is a miniature of engineering on a constrained platform. The case also suggests a direction for a local project: Taiwan's agricultural settings (orchards, paddy fields, greenhouses) face the same need to detect pests and disease, and the safety and low cost of a micro drone make it better suited than a large drone to working among densely planted crops.

Implementation Activities

Implementation 25-1 Simulating Mapping with a Micro Drone

Purpose: to experience in full, by simulation alone, the core logic of low-power SLAM—from mapping with sparse sensing, through observing the drift, to correcting by loop closure—and to carry out an engineering trade-off analysis of power against the number of sensors. This activity needs only Python and suits every level from high school to the undergraduate.

Part One: Build the Simulated Environment and Map Ideally (everyone)

1. Define a 2D room in Python: represent the walls and obstacles by line segments and place in it a virtual micro drone (represented by a position and a heading).
2. Implement distance sensing in four directions: cast a ray forward, backward, left, and right from the drone's position and compute the distance to the intersection with the nearest wall. This simulates the four ToF matrix sensors of Section 25.4.

3. Move the drone along a prescribed route (once around the room, say), recording the four distances at every step and drawing the detected wall points onto the map by the coordinate transformation of Chapter 10.
4. Display the growing map with matplotlib and watch the outline of the room emerge. This stage assumes the position is perfectly accurate, and the map should be quite handsome.

Part Two: Drift and Loop Closure (everyone)

1. Add odometry drift: put a small random error into the drone's estimate of position and angle at every step (a standard deviation of 2 centimeters in position and 0.5 degrees in angle, say) and build the map again.
2. Observe the pathologies: what has gone wrong with the map? Have the wall lines doubled? After a complete circuit, do the end and the start fail to meet? Measure how large that discrepancy is. Compare with the three map pathologies of Section 13.7.
3. Implement a simplified loop closure correction: when the drone returns near the start, add the constraint that the end should equal the start and spread the accumulated error evenly over every step of the whole trajectory (the method of Implementation 12-1 in Chapter 12), then draw the map again.
4. Quantify the improvement: compare the discrepancy between end and start before and after the correction, together with the visual quality of the map, compute the percentage by which the error fell, and set it against the 67 percent of the NanoSLAM paper.

Part Three: An Engineering Trade-off Analysis (undergraduate; the distinctive part of this chapter)

1. An experiment on the number of sensors: change the four-direction sensing to 8, 16, and 32 directions and compare the quality of the map (the completeness of the walls, the resolution of detail). Find the turning point beyond which the improvement in quality slows.
2. Estimate the power: supposing each ToF sensor draws 20 milliwatts and the power of each scan match grows linearly with the number of sensed points, compute the total power for each number of sensors.
3. Compute the flight time: by Formula One, with the battery energy fixed and flight itself drawing 5 watts, compute the flight time for each configuration and draw the three-way trade-off among the number of sensors, the quality of the map, and the flight time.
4. A decision exercise: if the task requires at least 5 minutes of flight and a map error below 10 centimeters, which configuration would you choose? Explain why.

Part Four: Further Exploration (advanced)

1. An experiment on the mapping rate: change the rate at which SLAM runs from once per step to once every three steps and once every five, and observe how much the map quality falls and how much power is saved. This corresponds to the dynamic frequency scaling strategy of a real system (Section 25.3).
2. Swarm simulation: add a second and a third virtual drone exploring separately, merge their maps afterwards, and compare the number of steps needed to cover the whole area by collaborative exploration against a single drone. This answers to Section 25.5 and Chapter 27.
3. A localization mode: build the map with one drone first, then let a second only localize within the known map (without mapping) and compare the

computation of the two. Set this against the difference between localization at 23 mW and mapping at 87.9 mW in Table 25-2.

Discussion Questions

1. What pathologies appeared in the map once drift was added? How large was the discrepancy at the end of a circuit? Does this agree with the description in Section 13.7?
2. The loop closure correction spreads the error over the whole trajectory rather than correcting the last step alone. Why is spreading it better? (Hint: the case in Chapter 12.)
3. Going from 4 sensors to 32, by how much did the quality of the map improve? By how much did the power rise? What number do you think most suitable, and why?
4. If you were the system designer of a micro drone, how would you trade a better map against a longer flight time? Does the trade-off differ from task to task?
5. (Undergraduate) By Amdahl's law (Formula Two), if 95 percent of ICP can be parallelized, by how much can GAP9's 9 cores speed it up? How far is that from the ideal 9 times, and where does the difference come from?

Extended Challenge (university level)

First: implement a simplified pose graph optimization—take the pose at every step of the simulation as a node, the motion between adjacent steps as an edge, and the loop closure as an extra edge, and optimize the whole trajectory with scipy's least-squares solver (the programmed version of Implementation 12-1 in Chapter 12), then compare its accuracy with the simple method of spreading the error evenly. Second: if your school has a Crazyflie or a similar micro drone platform, try reading its sensor data and

running your own SLAM program on a computer, and experience the difference between real data and simulation—real sensor noise, time delays, and the disturbances of flight will give you a deeper feeling for the engineering challenges of this chapter.

Further Reading

1. V. Niculescu et al., "NanoSLAM: Enabling Fully Onboard SLAM for Tiny Robots," IEEE Internet of Things Journal, 2023 (arXiv:2309.12008). (The principal source for this chapter; the code is public on GitHub at ETH-PBL/NanoSLAM.)
2. H. Müller et al., "GAP9Shield: A 150GOPS AI-capable Ultra-low Power Module for Vision and Ranging Applications on Nano-drones," arXiv:2407.13706, 2024. (The sensing and computing module of a micro drone, with measured power data.)
3. D. Rossi et al., the PULP platform papers and the official site pulp-platform.org. (The technical source of the parallel ultra-low-power computing paradigm.)
4. The official Bitcraze Crazyflie documentation (bitcraze.io). (Hardware specifications, development environment, and community resources for this chapter's experimental platform.)
5. GreenWaves Technologies GAP9 product documentation. (Specifications and development tools for the RISC-V low-power processor.)
6. P. Warden and D. Situnayake, TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers, O'Reilly, 2019. (The classic introduction to TinyML.)
7. L. Crupi et al., "Training on the Fly: On-device Self-supervised Learning aboard Nano-drones within 20 mW," arXiv:2408.03168, 2024. (On-device learning aboard a micro drone, the advanced direction of Section 25.6.)

Summary

1. Running SLAM on a micro robot meets three constraints—computation, memory, and electrical power; the traditional practice is to offload the computation to external facilities, but this brings dependence on infrastructure, communication latency, bandwidth and power costs, and problems of reliability, so that the robot is never truly autonomous.
2. NanoSLAM is a lightweight and optimized end-to-end SLAM method designed to run on centimeter-sized robots within a power budget of only 87.9 milliwatts; it was deployed on a micro drone weighing 44 grams and carrying GAP9, a RISC-V low-power parallel processor, attaining a mapping accuracy of 4.5 centimeters and an end-to-end execution time under 250 milliseconds.
3. GAP9 is built on the parallel ultra-low-power (PULP) computing paradigm, trading multi-core parallelism for efficiency at a power below 180 milliwatts; it supports dynamic scaling of clock and voltage and automatic clock gating, with an idle power as low as 45 microwatts. The openness of RISC-V makes it customizable and free of licensing cost.
4. NanoSLAM replaces the LiDAR with four ToF matrix sensors, matches scans with a parallelized ICP (55 milliseconds onboard), and corrects drift by pose graph optimization (trajectory error reduced by up to 67 percent); its three strategies for keeping light are parallelize rather than simplify, sparse rather than dense, and divide the labor rather than do everything.
5. A micro drone fits in the palm of the hand, suits indoor spaces, and is safe around people; onboard SLAM is a necessary condition for swarm robots to scale; and the constraints on AR glasses are astonishingly like those on a micro

drone, which makes low-power SLAM a shared requirement of every constrained platform.

6. Edge AI and TinyML join with SLAM in three ways: enhanced perception (YOLO can complete an inference below 100 milliwatts in as little as 17 milliseconds), learning in place of computation, and online self-learning; the central methodology is the co-design of hardware and algorithm.
7. Simulating mapping with a micro drone suits a high school project admirably: the idea is intuitive, it can be simulated completely, and its levels are distinct; the recommended teaching rhythm is to build the perfect map first, then add drift deliberately to break it, and finally repair it by loop closure.

Carrying Forward

This chapter has compressed SLAM into a milliwatt budget and measured three sets of key numbers along the way: localization needs far less density of measurement than intuition suggests—32 ToF rays already come close to saturation (0.030 meters), and doubling them to 64 buys nothing; but range matters far more, and widening it from 1 meter to 4 improves the error nineteenfold. In hardware, at a fixed throughput many cores running slowly can bring the power down to 4.0 percent, but past about 33 cores leakage takes its revenge; on real chips GAP9's efficiency is 39 times that of the STM32. And at the level of the platform, the weight of a compute module affects endurance 4.7 to 10.6 times as much as its power does—in choosing a board the first question is how much it weighs, not how many watts it draws.

This chapter has displayed SLAM realized on a platform under extreme constraint, and has brought out an idea that runs through the whole book: the bottleneck in

performance lies more often in a method not yet optimized than in a want of computing power. The next chapter focuses on the challenges of drone navigation itself—how is one to fly in a GPS-denied environment? How are fast motion, vibration, and motion blur to be handled? How do optical flow, visual-inertial odometry, and visual SLAM divide the labor on a drone? Chapter 26 carries the discussion of Part VII to the heart of flight.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 26

Drone SLAM and Visual-Inertial Navigation

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 25 compressed SLAM into a milliwatt computing budget. This chapter changes the dimension: speed.

The typical speed of a ground robot is half a meter per second, and a drone may be ten times that or fifty. The difference is not merely running the same method faster—it blurs into a smear the clean corners on which Chapter 20 lives, it leaves no time at all to establish the mid-term data association on which Chapter 22 depends, and it has the propellers manufacture, in a way one would not expect, the very IMU bias discussed in Chapter 7.

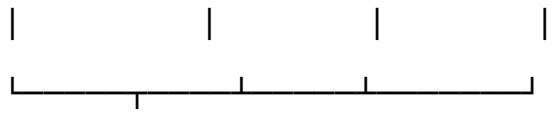
The three figures of this chapter measure exactly these things: Figure 26-1 compares the drift behavior of the three tiers—optical flow, VIO, and VI-SLAM; Figure 26-2 takes apart and measures the triple blow of Section 26.4 and finds which remedy is the most effective; and Figure 26-3 explains something counterintuitive—that raising the IMU sampling rate cannot save you from the aliasing vibration causes.

Figure 26-0 The Link Map of Chapter 26

The foundations built up so far

Chapter 7 The IMU and bias Chapter 20 Features Chapter 23 Visual-inertial SLAM

Chapter 25 Micro platforms



▼

【Chapter 26 The new problems flight brings: fast, shaking, and hard to see】

GPS-denied ─ optical flow / VIO / VI-SLAM ─ motion blur ─ vibration ─ graceful degradation

▼

Chapter 27 Multi-robot collaboration and the cloud-edge division of labor (closing Part VII)

▼

Part VIII ROS implementation　　Part IX AI and semantic navigation

Learning Objectives

In the last chapter we saw how SLAM is compressed into a milliwatt budget. This chapter focuses on the challenges of drone navigation itself: in the air the robot loses the support of the ground, possesses a full six degrees of freedom, moves far faster, and may be brought down by any error in localization. After studying this chapter, students should be able to:

1. Explain the four particular difficulties of localizing a drone as against a ground robot.
2. Explain how GPS-denied environments arise and what the typical settings are, and list the means of localization available.
3. Compare the levels of capability of optical flow, VIO, and visual SLAM, and where each is suitable.
4. Explain how fast motion, vibration, and motion blur affect visual localization, and propose remedies.

5. Plan the procedure and the safety measures for an indoor drone navigation experiment.
6. Explain the properties and the uses of the two great visual-inertial datasets, EuRoC and TUM-VI.
7. Explain the line of technical development from the micro drone to the vision-navigated drone.

Core Concepts

中文名詞	English Term	Explanation
微型飛行器	MAV	A small unmanned aircraft, the principal platform of visual-inertial navigation research.
GPS 拒止環境	GPS-denied	An environment where the satellite signal is blocked or jammed and cannot be used.
光流	Optical Flow	The vector field of the brightness pattern in an image moving over time.
光流定位	Optical Flow Positioning	Estimating horizontal velocity from the optical flow of a downward camera to hold a hover.
視覺慣性里程計	VIO	Coupling camera and IMU tightly to estimate motion; it accumulates drift.
視覺慣性 SLAM	VI-SLAM	VIO plus mapping and loop closure, which removes long-term drift.
動態模糊	Motion Blur	Blur in an image caused by the camera moving during the exposure.
特徵軌跡縮短	Shortened Feature Tracks	At high speed a feature leaves the field of view quickly and is tracked for a

中文名詞	English Term	Explanation
		shorter time.
滾動快門	Rolling Shutter	The sensor exposes row by row, which distorts the image at high speed.
全域快門	Global Shutter	The whole frame is exposed at once, which suits fast motion.
積極機動	Aggressive Maneuver	A flight maneuver of large angle and high acceleration.
光度校正	Photometric Calibration	Calibrating the camera's brightness response and vignetting; important for direct methods.
動作捕捉系統	Motion Capture	Laboratory equipment supplying millimeter-level true poses from external cameras.

26.1 The Particular Difficulties of Localizing a Drone

Section 3.6 compared the navigation requirements of five platforms; this section goes deeper into why the drone is the most severe of them.

Four Particular Difficulties

1. A full six degrees of freedom: a ground robot moves on a plane and has only three degrees of freedom (the x, y, θ of Section 10.3); a drone may translate and rotate as it pleases in three-dimensional space. With more states and more complex motion, the difficulty of estimation rises at once.
2. No odometry: this is the point most easily overlooked and yet absolutely crucial. A wheeled robot has wheel encoders supplying odometry (Chapters 3 and 13), which is a steady reference for motion even where it is imperfect; a drone in the air touches nothing and has no equivalent sensor at all. It can only integrate the IMU—and Section 7.3 told us that an IMU used alone drifts quickly.
3. High speed and violent dynamics: a drone flies far faster than an indoor ground robot, and its attitude changes violently. This leads directly to motion blur and to difficulty in tracking features (Section 26.4).
4. The consequences of an error are severe: a wheeled robot that localizes wrongly will at worst bump into a wall and stop; a drone that localizes wrongly may fall out of the sky, damaging the equipment and even injuring people. This makes the demand on the robustness of the system far higher than the demand on its accuracy.

The Double Constraint of Payload and Power

Chapter 25 set out the problems of a constrained platform in detail, and for a drone

they are sharper still: every additional gram of sensor weight consumes extra thrust and extra power, and flight itself has already consumed the overwhelming part of the charge. This constrains the choice of sensors on a drone severely—and it is the fundamental reason Section 9.5 called a camera together with an IMU the ideal combination for a drone.

The Severe Demand on Timeliness

If a ground robot's localization is half a second late, it merely stops half a beat behind; if a drone's localization is half a second late, it may already have moved several meters in flight, and the control loop loses its stability.

A drone system therefore tolerates latency very poorly. This explains why Chapter 25 said NanoSLAM must press its end-to-end execution time within 250 milliseconds, and why the attitude control loop of a high-grade flight controller runs at hundreds of hertz—which is exactly the value of the IMU's high-rate output in Chapter 7.

26.2 Navigation in GPS-Denied Environments

Chapter 5 discussed the principles and the limitations of GNSS in detail; this section focuses on the drone's position when the satellite signal is unavailable, and what can be done.

Typical GPS-Denied Settings

- Indoors: building materials block the satellite signal completely (Section 5.5). Inspection and stocktaking in factories, warehouses, shopping centers, and gymnasiums all belong here.
- Beneath bridges and structures: bridge inspection is an important drone application, but the deck itself is the obstruction.

- Tunnels and underground spaces: mines, sewers, underground car parks—no satellite signal at all.
- Urban canyons: the corridors between tall buildings mentioned in Section 5.5 not only block the signal but suffer severe multipath that produces a wrong position—which is more dangerous than no signal at all, because the system may not know that it is wrong.
- Under forest and vegetation: the canopy makes the signal weak and unsteady.
- Deliberate interference: the jamming and spoofing mentioned in Section 5.5 are a serious security question for a drone.

The Means of Localization Available

In a GPS-denied environment the means of localization a drone can rely on, ordered by reliability, are as follows:

Means	What it supplies	Limitations	Chapter in this book
IMU	High-rate attitude and motion	Drifts quickly when used alone	Chapter 7
Barometer	Relative height	Affected by air currents and temperature	This chapter
Optical flow (downward camera)	Horizontal velocity	Needs ground texture; supplies no absolute position	Section 26.3
Ranging sensor (downward)	Height above the ground	Unevenness of the ground causes misjudgment	Chapter 8
VIO (camera +	A full 6-DoF pose	Drifts; needs texture	Chapter 23

Means	What it supplies	Limitations	Chapter in this book
IMU)		and light	
VI-SLAM	Pose + map + loop closure	Heavier computation	Chapter 23
UWB / visual markers	An absolute correction of position	Infrastructure must be deployed beforehand	Chapter 4

Table 26-1 Means of Localizing a Drone in a GPS-Denied Environment

A Layered Strategy

A practical system combines these means in a layered design:

1. The bottom layer (hundreds of Hz): the IMU supplies attitude and keeps the flight stable. This layer may not be interrupted at all, or control is lost outright.
2. The middle layer (tens of Hz): optical flow and ranging supply velocity and height and sustain a hover in place. Even with no map the drone can stay where it is without drifting away.
3. The upper layer (10 to 30 Hz): VIO or VI-SLAM supplies a full pose and a map and sustains path planning and autonomous navigation.
4. The correcting layer (at irregular intervals): UWB, a visual marker, or a return to a region where GPS is available supplies an absolute correction of position (the fusion idea of Chapters 4 and 5).

The importance of this layering is fault tolerance: if the VIO of the upper layer fails, the drone can still hover on the optical flow of the middle layer and need not fall; and if the middle layer fails too, the IMU at the bottom can at least keep the attitude steady so that the operator may take over. The central principle of safety design is to let failure

degrade step by step rather than collapse in an instant.

26.3 Optical Flow, VIO, and Visual SLAM

This section sorts out the levels of capability of three visual localization technologies—they are often used interchangeably, but what they supply differs greatly.

Optical Flow: the Most Basic Layer

Section 9.4 introduced optical flow as the movement of the brightness pattern in an image over time. On a drone it has one extremely practical application: downward optical flow positioning.

The practice is to fit a downward-facing camera to the belly of the aircraft and compute continuously the speed at which the texture of the ground moves across the frame. Together with the height above the ground supplied by a ranging sensor, this converts into the drone's horizontal speed.

- What it supplies: horizontal velocity (not position).
- What it can do: hold a hover. If the optical flow shows the ground moving backward, the drone is drifting forward, and the flight controller corrects backward. This lets a drone hold its place steadily indoors with no GPS.
- Its limitations: it gives only velocity, which must be integrated to give position, so it still drifts; it needs the ground to have texture enough (over a plain white floor or water it fails); and an inaccurate measurement of height produces a wrong conversion of speed directly.

Optical flow positioning is standard equipment for the indoor hover of a consumer drone, costs very little computation, and is a good example of solving the most crucial

problem with the fewest resources.

VIO: a Full Six-Degree-of-Freedom Odometry

VIO couples camera and IMU tightly (Sections 7.6 and 23.3) and outputs a full 6-DoF pose and velocity. What it supplies is far more complete than optical flow, enough to sustain path following and autonomous flight.

But VIO is in essence still an odometry—it accumulates drift (the property of visual odometry noted in Section 20.1). Let the flight go on long enough and the estimated position departs from the true one.

Visual-Inertial SLAM: Adding a Map and Loop Closure

VI-SLAM adds mapping and loop closure detection on top of VIO (Chapter 23), and can correct the accumulated drift on revisiting the same place. The price is a rise in computation and in memory.

Technology	Output	Does it drift	Computation required	Typical use
Optical flow	Horizontal velocity	Yes, once integrated	Very low	Holding a hover
VIO	6-DoF pose + velocity	It accumulates	Moderate	Path following, autonomous flight
VI-SLAM	Pose + map + loop closure correction	Loop closure corrects it	Higher	Exploratory mapping, long tasks

Table 26-2 A Comparison of the Capabilities of the Three Visual Localization Technologies

How to Choose

- If the task is to hover and film: optical flow is enough, and saves both power and computation.
- If the task is to fly a prescribed route: VIO is needed to track position.
- If the task is to explore an unknown space and map it: VI-SLAM is needed.
- If the task is to inspect the same site repeatedly over long periods: VI-SLAM together with map reuse (the multi-session working of Atlas in Chapter 23) suits best.

This way of thinking about levels answers to a principle this book stresses repeatedly: do not use technology beyond what the task requires, because on a drone every ounce of computation and power is paid for in flight time.

Measuring It: How Much Does Each of the Three Tiers Drift

Table 26-2 distinguished the three technologies by the column headed does it drift, but did not say how much, or in what form. This subsection measures it with one complete flight simulation.

The setting is a drone flying six circuits of an elliptical route over 72 seconds at an average speed of about 2.5 meters per second. Each of the three estimators is modeled on its own physical properties: optical flow measures horizontal velocity in the body frame, which must be multiplied by the height measured by a ranging sensor to convert into true velocity (the height carries a 4 percent error) and then rotated back into the world frame by a heading integrated from the gyroscope (the heading drifts); VIO outputs relative poses directly and its error accumulates step by step; VI-SLAM sits on top of VIO and, on the completion of every circuit, forms a loop closure with the same position on the circuit before and performs pose graph optimization. Each setting is

run five times and the median taken.

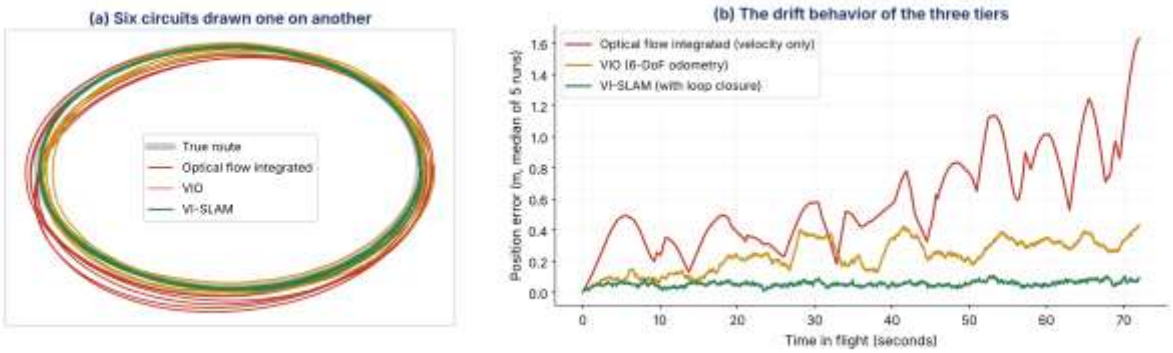


Figure 26-1 The Drift Behavior of Optical Flow, VIO, and VI-SLAM

Panel (a) is the six circuits drawn one on top of another. The green of VI-SLAM nearly coincides with the true route, and the six circuits drawn together show no divergence; the amber of VIO wanders outward circuit by circuit; and the red of optical flow wanders more plainly still, and in no fixed direction—which is what comes of a heading error turning the whole velocity vector.

Panel (b) is the position error against time. Over the second half (after 36 seconds) the mean errors are 0.810 meters for optical flow, 0.296 meters for VIO, and 0.060 meters for VI-SLAM; the peaks are 1.63, 0.43, and 0.11 meters. VI-SLAM is 5 to 13 times better than the other two.

But what deserves attention is the shape of the three lines rather than their height. The green line is flat from beginning to end—its error does not grow with the time in flight but is held by loop closure at a fixed level. The red and amber lines show a plain sawtooth: around each circuit the error first rises and then partly falls back. It falls back not because it has been corrected but because the route is closed, so that on coming round near the start the accumulated error happens to cancel in part—a coincidence, not a mechanism. Were the route one-way, these two lines would only climb.

This figure also shows the ground of those few recommendations under How to Choose in Section 26.3. Hovering requires no position but only velocity, so the drift of optical flow is no trouble at all; but let the task require flying to a given coordinate or coming back when done, and a scheme without loop closure will accumulate an error of meters within tens of seconds.

Why Optical Flow Is Worse Than VIO

On the face of it, an optical flow measurement is simpler and its error should be smaller. But it has two weaknesses VIO does not.

First, it gives only velocity and no attitude at all, so that turning the velocity into the world frame must borrow a heading integrated from the gyroscope—and an error in heading rotates the whole velocity vector directly, producing a position error proportional to the distance flown rather than to the square root of the time.

Second, its conversion of velocity depends on the measurement of height. This simulation gave the height only a 4 percent error, and that alone produced a 4 percent error of scale throughout. Should the drone fly over a table, the height the ranging sensor measures suddenly shrinks and the optical flow misjudges it as flying more slowly—which is the commonest cause of indoor drift in a consumer drone.

26.4 Fast Motion, Vibration, and Motion Blur

This section is the most central technical challenge of visual localization on a drone.

The Triple Blow

Research indicates that existing VIO systems degrade at high speed because of

motion blur, shortened feature tracks, and increased latency in the control loop. These three happen to form a vicious circle:

1. Motion blur: the camera moves during the exposure and the image blurs. At such speeds new problems emerge and the existing difficulties are aggravated—motion blur becomes more marked and damages the tracking of visual features. In a blurred image a corner becomes a smear and feature extraction fails outright (the features of Chapter 20 need clean corners).
2. Shortened feature tracks: in fast flight the same feature point sweeps across the field of view and is gone. The mid-term data association stressed in Section 23.2 is thus hard to establish, the system can rely only on short-term association, and accuracy falls.
3. Latency in the control loop: the visual processing itself takes time, and at high speed that delay corresponds to a larger displacement, so that the quality of control deteriorates and oscillation may set in.

What the Datasets Show of Reality

These challenges are plainly displayed on the standard benchmark datasets. Research has observed that because of the drone's fast motion the images are somewhat blurred, and markedly so in the difficult sequences MH_04, MH_05, V1_03, and V2_03.

More interesting still is one detail: the machine-hall sequences of EuRoC all begin with fast motion to assist IMU initialization (Section 23.4), but the motion blur this produces is a great challenge to visual tracking; the authors of DSO therefore cut off the beginning and use only the part where the drone is in the air, whereas for a VIO system IMU initialization is indispensable and the beginning cannot be discarded.

This detail shows two things at once. First, IMU initialization needs excitation of motion (as Section 23.4 stressed repeatedly), but the excitation itself causes blur, so that the two pull against each other. Second, different methods may be evaluated under different conditions on the same dataset, and students should be alert to such details when reading papers and comparing figures.

The Order of Magnitude of Speed

To give students a concrete idea of what fast means: the EuRoC micro air vehicle dataset seldom exceeds 5 meters per second; a racing drone dataset may reach 28 meters per second, but usually with RGB cameras on a structured indoor course or a short outdoor track. Research has supported up to 17.5 meters per second with a stereo RGB VIO, but it depends on stereo vision and presumes favorable lighting.

By contrast, an indoor ground robot usually moves below half a meter per second—the drone is ten times faster, or tens of times. This is why visual localization on a drone is so much harder.

Five Remedies

1. Raise the shutter speed: a shorter exposure reduces blur directly, at the price of a darker image, which calls for a better sensor or a brighter environment.
2. Use a global shutter: a rolling shutter exposes row by row and skews the image under fast rotation; a global shutter exposes the whole frame at once and is a necessary choice for high-speed applications.
3. Lean on the IMU: research has integrated an IMU-based motion prediction module to improve feature tracking across frames, particularly during fast motion. The IMU can predict where a feature ought to have moved to, which

narrows the search greatly and raises the rate of successful tracking. This is the concrete realization of Section 7.6's the IMU supplies vision with an initial value.

4. Enhance the image: preprocess a blurred or low-contrast image. Research has combined contrast-limited adaptive histogram equalization (CLAHE), gradient filtering, and descriptor verification to maintain stable feature tracks under motion blur.
5. Adaptive weighting: adjust the relative trust in vision and inertia dynamically according to image quality. Research has validated an adaptive covariance method on the EuRoC and TUM VI datasets and in field tests, markedly improving a drone's navigation accuracy in situations with motion blur.

The Problem of Vibration

Besides the speed of flight, the high-frequency vibration caused by the propellers and motors is a problem peculiar to drones:

- On the IMU: vibration contaminates the accelerometer readings, and if its frequency approaches the sampling rate it may even cause aliasing. In practice mechanical isolation (soft rubber mounts) and filtering are required.
- On the camera: the slight blur caused by high-frequency vibration lowers the quality of features, and is usually relieved by a damped mount and a faster shutter.
- On the mounting: if the relative position of sensor and airframe (the extrinsics, Chapter 10) works loose and changes through vibration, the accuracy of the whole system suffers.

Measuring It: How Much Does Each of the Triple Blows Take Away

Section 26.4 gathered the difficulties of fast flight under three heads: motion blur,

shortened feature tracks, and latency in the control loop. This section measures the first two separately, and checks along the way which of the five remedies is the most effective.

The method of the experiment: synthesize an image of the ground carrying texture at several scales together with geometric corners (simulating brick joints, floor markings, and a door frame), compute the corresponding pixel displacement from speed × exposure time × focal length ÷ depth, apply a linear motion blur, then detect with ORB and cross-match, counting how many features can still be matched correctly against the clean image. The camera parameters are those of the wide-angle configuration common on drones (a focal length of 380 pixels), with a scene depth of 4 meters.

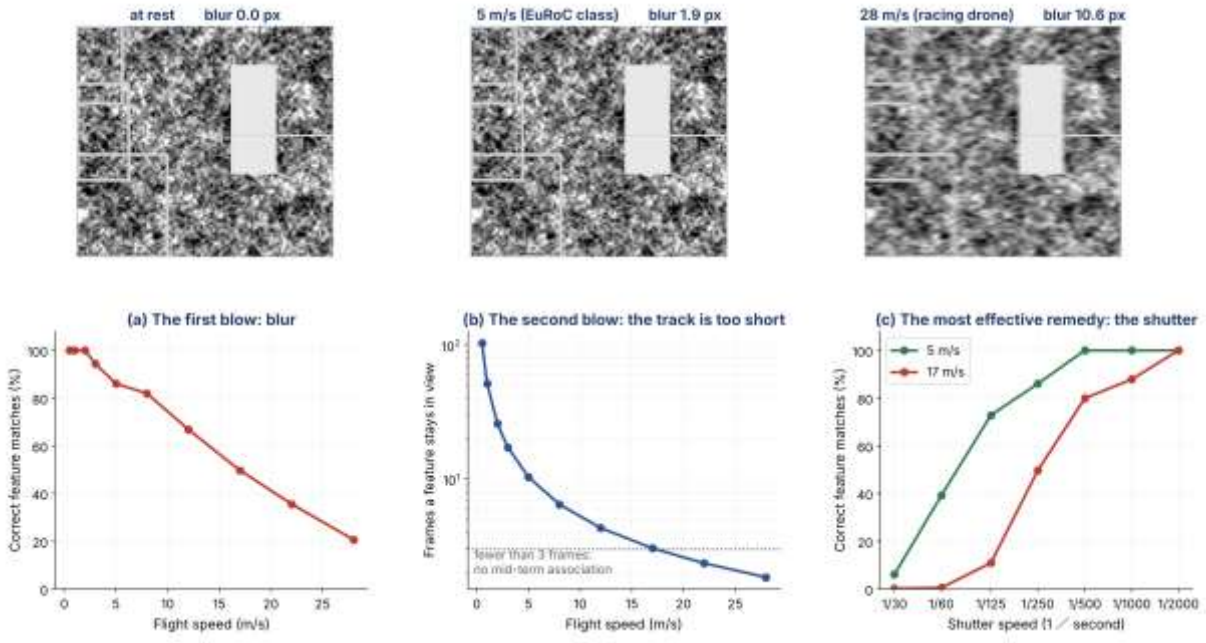


Figure 26-2 The Triple Blow of Speed, and the Most Effective Remedy

The top row is the same patch of texture at three speeds. At rest the corners are clean; at 5 meters per second (the typical speed of the EuRoC micro air vehicle dataset) the blur is 1.9 pixels and the edges can already be seen to soften with the naked eye; at 28 meters per second (a racing drone) the blur is 10.6 pixels, the fine texture has all but

vanished, and only large patches of light and dark remain.

Panel (a) quantifies the first blow. Within 2 meters per second the feature matching rate is still 100 percent; at 3 meters it is 94 percent, at 5 meters 86 percent, at 12 meters 67 percent, at 17 meters 50 percent, and at 28 meters only 21 percent remains. Note that the turning point falls between 2 and 5 meters per second—which is exactly the cruising speed of an ordinary indoor drone, and explains why the difficult sequences of EuRoC are precisely those flown faster.

Panel (b) quantifies the second blow, and this one is the more deadly. The vertical axis is how many frames a feature point can be seen in from entering the field of view to leaving it (the camera runs at 30 fps). At half a meter per second it is 104 frames (3.5 seconds); at 2 meters 26 frames; at 5 meters 10 frames; at 17 meters only 3; and at 28 meters—only 2.

Two frames is what? It means the feature is detected in this frame and is already off the screen in the next but one. With barely a second observation there is no reliable triangulation (Section 21.5 requires parallax), no mid-term data association (Section 23.2), and no sound way to create a map point. In other words, at 28 meters per second visual SLAM can hardly proceed even if the images are perfectly clean. Blur merely adds frost to the snow.

Panel (c) tests the first remedy mentioned in Section 26.4, raising the shutter speed. Take 17 meters per second: at a shutter of 1/30 second the matching rate is 0.2 percent, at 1/60 0.5 percent, at 1/125 10.9 percent, at 1/250 49.7 percent, at 1/500 79.9 percent, at 1/1000 87.9 percent, and at 1/2000 100 percent. Going from 1/125 to 1/500 takes the matching rate from one tenth to eight—the cheapest of the five remedies and the

one that acts most immediately.

The price is there too, of course: double the shutter speed and half the light is lost. High-speed visual localization on a drone is therefore in practice an optical problem—either the environment must be bright enough, or one must change to a fast lens and a sensitive imager. This is also why research on racing drone vision is mostly confined to a structured indoor course or an outdoor track in good light.

> **A Practical Suggestion for a Project**
>
> Setting panels (a) and (b) side by side gives a very clear upper limit on speed: within 2 meters per second both the blur and the track length are still healthy (100 percent matching, 26 frames and more); 5 meters per second is the usable boundary (86 percent, 10 frames); and beyond 10 meters per second one must deal with the shutter, the light, and the algorithm all at once.
>
> In planning a drone project, rather than pursue speed, students should first hold the speed within 2 or 3 meters per second and make the localization and the mapping steady; speed is the last parameter that should be relaxed.

Measuring It: What Does the Vibration of the Propellers Turn Into

Section 26.4 ended by noting that vibration contaminates the accelerometer readings, and that if its frequency approaches the sampling rate it may even cause aliasing. That sentence deserves unfolding, because the consequence of aliasing is far more serious than noise growing larger.

The experimental setting: superpose propeller vibration (a fundamental plus a second harmonic, of amplitude 2.5 meters per second squared) on a slowly varying true acceleration signal, sample at 400 Hz or at 1000 Hz, and compare the mean difference

between the sampled signal and the truth—a mean difference which, in the eyes of the estimator downstream, is simply an accelerometer bias.

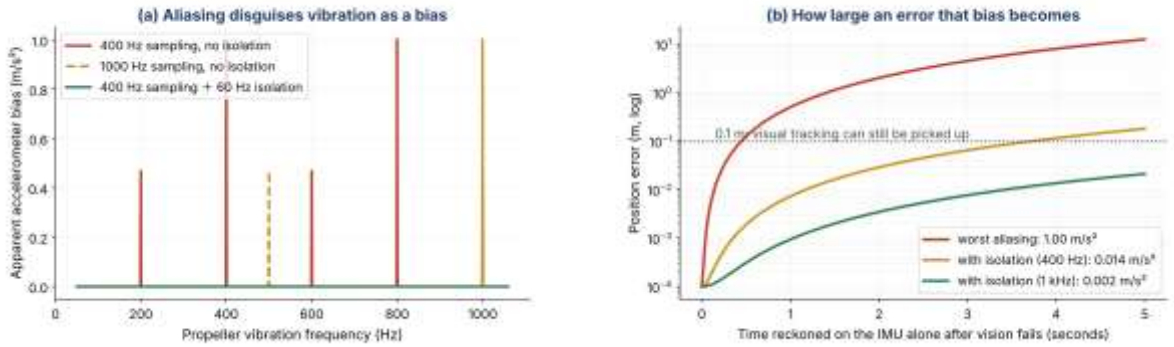


Figure 26-3 The Aliasing of Vibration, and Why It Disguises Itself as a Bias

Panel (a) sweeps the vibration frequency. The red line is 400 Hz sampling with no isolation: at most frequencies the apparent bias is near zero, but at 200, 400, 600, and 800 Hz it leaps suddenly to 1.00 meters per second squared. These are exactly the integer and half-integer multiples of the sampling rate—at these frequencies the vibration is folded down into a DC component, which is to say a constant offset that looks exactly like a bias.

The amber dashed line raises the sampling rate to 1000 Hz. The positions of the spikes change, but their height does not change at all—still 1.00 meters per second squared. This is the most important message of the figure: raising the sampling rate cannot cure aliasing; it can only move the problem to other frequencies. This is a fairly common misunderstanding.

The green line adds mechanical isolation with a cutoff at 60 Hz (a soft rubber mount plus an analog low-pass). The greatest apparent bias falls from 1.00 to 0.014 meters per second squared, some seventy times smaller. The key is that this filtering happens before the sampling—the high-frequency content is stopped before it ever reaches the

analog-to-digital converter, and so never has the chance to be folded down. Digital filtering after sampling is wholly useless against aliasing, because the signal is already spoiled.

Panel (b) shows how serious that 1.00 meters per second squared is. A bias integrated twice is a position error: a visual outage of 1 second produces an error of 0.50 meters, of 2 seconds 2.0 meters, and of 4 seconds 8.0 meters. Set beside the conclusion of Figure 23-1 in Chapter 23—that with the bias estimated to ±0.01 the error after 2 seconds is only 0.025 meters—this false bias from vibration is eighty times worse than the best case discussed there.

More troublesome still, this offset is not random but changes with the rotor speed. As the drone accelerates the rotor speed rises and the vibration frequency moves with it, so that the false bias may leap from zero to 1.00 and drop back to zero. The IMU bias estimation of Section 23.4 assumes that the bias varies slowly, and against a jump of this kind it is wholly powerless. Isolation is therefore not a detail of tuning but a precondition of the system working at all.

A Check That Can Be Done in the Laboratory

Fix the drone to a table (with the propellers removed or a guard fitted), raise it slowly from idle to full throttle, and record the raw accelerometer readings throughout.

Then compute the mean over each band of rotor speed. If the mean departs plainly from gravity at a few particular speeds, that is aliasing—and you have just found the bands of rotor speed that must be avoided.

A fuller practice is to take a fast Fourier transform of the raw signal, see where

the propellers' fundamental and harmonics fall, and choose the softness of the damped mount (which is to say its cutoff frequency) accordingly. This experiment needs only a drone and a recording program, and can save weeks of debugging afterwards.

26.5 An Indoor Drone Navigation Experiment

This section explains how to plan a drone indoor navigation experiment that is both safe and instructive, which is also the foundation of a project.

Safety First: What Must Be Said Before Anything Else

The greatest difference between a drone experiment and a ground robot experiment is the risk. The following principles are not to be compromised:

- A net or a cage: an indoor experiment should be conducted in a space with a safety net, or with a model whose propellers are guarded by a frame.
- Choose a micro platform: the micro drones introduced in Chapter 25 weigh only tens of grams, and even out of control and striking a person will scarcely cause injury. For a teaching experiment, starting from a micro platform is strongly recommended.
- An emergency stop: the operator must be able to cut the power with one key at any moment, and this function must not depend on the autonomous system that is being debugged.
- Test in stages: verify in simulation first, then test on a tether, and only then fly freely. Skipping a stage is the principal cause of accidents.

- Where people stand: during the experiment everyone not needed should be outside the net, and the operator should stand behind the drone and not in front of it.

Preparing the Site

- Texture: the floor and the walls need visual texture enough (the curse of the white wall of Chapters 9 and 20 holds on a drone just as well). Patterned mats may be laid on the floor if necessary.
- Light: even and ample, avoiding backlight and strong contrasts of light and dark. A window may cause overexposure, and drawing the curtains to keep the lighting steady is advised.
- Air currents: avoid air conditioning blowing directly or a draft through an open window, which disturbs the stability of a micro drone severely.
- A ground truth system (optional): a motion capture system supplies millimeter-level true poses for quantitative evaluation (Section 26.6). Failing that, the AprilTag of Chapter 4 can supply an approximate ground truth (the fourth use in Section 24.3).

The Procedure of the Experiment

1. Check the system: confirm that the IMU calibration, the camera calibration (Section 9.2), the sensor extrinsics (Chapter 10), and the communication link are all in order.
2. Test the hover: the most basic ability of all. Start optical flow positioning, let the drone hover in place for 30 seconds, and measure the drift in position. If the hover is not steady, there is no point attempting autonomous flight afterwards.

3. Initialize VIO: following Section 23.4, complete the IMU initialization with a motion carrying acceleration, deceleration, and rotation, and confirm that the scale estimate is correct (a known distance will verify it).
4. A simple route: set a simple square or straight route, let the drone fly it autonomously, and record the trajectory.
5. A loop closure test: let the drone go once around and return to the start, measure the discrepancy between end and start, and assess the accumulated drift (the evaluation method of Section 13.7).
6. A challenge test: raise the difficulty step by step—increase the speed, add sharp turns, cover the camera briefly—and observe under what conditions the system fails. This is the most instructive part of all.

Recording the Data

Every flight should be recorded in full: images, raw IMU data, flight controller state, estimated trajectory, and (if available) the true trajectory. These data can be analyzed offline again and again, and can serve as a dataset of your own making—for project research, a dataset of your own site is itself a valuable result.

26.6 Visual-Inertial Datasets: EuRoC and TUM-VI

Research on drone SLAM can be done without drone hardware—which is exactly the value of a public dataset. This section introduces the two great standard datasets.

The EuRoC MAV Dataset

Case 7-3 in Chapter 7 introduced EuRoC already; details are added here. It contains eleven sequences recorded on a micro air vehicle, supplying stereo camera images, IMU measurements, and true poses from a motion capture system.

Each sequence covers 30 to 60 seconds of flight, contains high-fidelity 6-DoF ground truth poses from external motion capture together with a high-resolution point cloud scan of the operating environment, so that both trajectory estimation and dense three-dimensional reconstruction may be evaluated against geometrically consistent reference data. The typical evaluation procedure passes 20 Hz monocular images and 200 Hz IMU data to a feature-based VIO (ORB-SLAM3 with loop closure turned off, say), producing a camera trajectory at true scale together with sparse three-dimensional landmarks.

- The classes of sequence: the machine hall (MH) sequences were recorded in an industrial plant, the room (V1, V2) sequences in a laboratory with motion capture.
- The grades of difficulty: every group of sequences is graded easy, medium, and difficult, the difficulty coming from speed, from light, and from how violent the motion is. This lets students test an algorithm step by step.
- Advice on use: start from MH_01_easy to build up an experience of success, then take on difficult sequences such as V1_03_difficult.

The TUM-VI Dataset

TUM-VI is another widely used benchmark. It supplies camera images of 1024 × 1024 resolution at 20 Hz with high dynamic range and photometric calibration; the IMU measures three-axis acceleration and angular rate at 200 Hz, and camera and IMU are time-synchronized at the hardware level.

By comparison the EuRoC MAV sequences are shorter and less varied, containing recordings of only one machine hall and one laboratory; EuRoC MAV moreover carries no photometric calibration, which matters considerably for evaluating direct methods

(Section 20.2).

- Its distinctive features: it covers a variety of scenes indoors and out, its sequences are longer, it has photometric calibration, and its time synchronization is done at the hardware level.
- Relevance to AR / VR: Section 23.7 quoted ORB-SLAM3's millimeter-level accuracy on the TUM-VI room sequences, which are exactly the room scenes of this dataset.

Property	EuRoC MAV	TUM-VI
Platform	Micro air vehicle	Handheld sensor rig
Images	Stereo, 20 Hz	Stereo 1024×1024, 20 Hz
IMU	200 Hz	200 Hz, hardware time synchronization
Ground truth	Motion capture / laser tracker	Motion capture (some sequences)
Photometric calibration	None	Yes
Variety of scenes	One machine hall + one laboratory	A variety of scenes indoors and out
Additional data	A point cloud scan of the environment	High dynamic range images

Table 26-3 A Comparison of the EuRoC and TUM-VI Datasets

Advice on Using Datasets

- Comparing algorithms: compare different systems on the same dataset (as in Implementation 23-1 of Chapter 23), and only then are the figures comparable.

- Mind the evaluation conditions: as Section 26.4 noted of whether the beginning is cut off, the conditions of evaluation may differ from paper to paper, and comparing numbers directly calls for caution.
- A dataset of your own: a public dataset cannot cover your particular site. Recording data at a university site by the method of Section 26.5 is a contribution of real value for a project.
- Newer datasets: datasets of greater speed and more sensing modalities have appeared in recent years, such as racing drone datasets (up to 28 meters per second) and datasets containing thermal imagery, which suit advanced research.

26.7 From NanoSLAM to the Vision-Navigated Drone

This section joins Chapters 24 and 25 together and traces the development of autonomous drone navigation and the directions ahead.

Three Levels of Drone Autonomy

Level	Capability	Key technology	Representative platform
Stable	Does not drift away; can hover	IMU + optical flow + ranging	Consumer drone
Autonomous	Can fly a route and avoid obstacles	VIO + path planning	Professional drone
Intelligent	Can explore, map, and understand a task	VI-SLAM + semantics + VLM	Research platforms, future products

Table 26-4 Three Levels of Autonomous Capability in a Drone

This table of levels can help students place the aim of their own project. Most teaching projects should confirm the foundations at the stable level and then take on the autonomous level; the intelligent level suits graduate work or a long-running project.

Three Main Lines of Technical Development

1. Computation settles downward: from computing on an external computer (offloading), to high-performance onboard computing (Jetson, say), to the milliwatt-level dedicated chip of Chapter 25. This line lets autonomous capability enter smaller and cheaper platforms.
2. Sensing grows lighter: from the heavy LiDAR, to a camera with an IMU, to the ToF matrix sensors of Chapter 25. This line lowers the threshold in weight and in power.
3. Intelligence rises: from geometric navigation, to semantic understanding (Chapter 24), to task-level autonomy driven by a VLM. This line raises the complexity of the task a robot can carry out.

It is worth noting that these three lines are converging: the micro platform of Chapter 25 can already run YOLO object detection (below 100 milliwatts, in as little as 17 milliseconds), which means a drone that is small, frugal, and capable of semantic understanding is becoming a reality.

Project Directions at a University Site

- Indoor survey: scan the interior of a departmental building with a micro drone (a high atrium, a stairwell) and produce a three-dimensional model. These are

spaces a ground robot can hardly reach, and they display the drone's distinctive value.

- Facility inspection: checking rooftop solar panels, exterior walls, and high pipework—a drone can safely take the place of a person working at height.
- Agricultural applications: answering to Case 25-3 in Chapter 25, Taiwan's orchards and greenhouses are ideal sites for a micro drone.
- A teaching dataset: record and publish a visual-inertial dataset of a university site for the students and researchers who come after. This is relatively easy to begin and yet yields value over a long time.
- Ground-air collaboration: let a robot dog (Chapters 18 and 24) and a drone explore together—the ground robot has long endurance and a large payload, the drone a wide view and the ability to go high. This is a concrete application of the multi-robot collaboration of Chapter 27.

Advanced Directions

A few research directions worth watching, for students minded to go deeper:

- Perception in fast flight: as Section 26.4 described, existing VIO degrades at high speed. Breaking the speed limit with unconventional sensors such as a thermal camera is an active direction of research.
- Joining learning to geometry: Section 20.7 noted that deep-learned features are entering SLAM. Research has integrated a deep feature extractor and a sparse feature matcher with traditional hand-crafted keypoint detection and optical flow tracking, accelerating the inference with TensorRT, and improved the performance of front-end tracking under fast camera motion and changing light.

- Learned inertial odometry: a learning method on the IMU alone can already approach the performance of VIO in certain situations—an important fallback where vision fails (darkness, smoke).
- Swarm collaboration: several micro drones mapping and exploring together is the subject of Chapter 27, and the last piece of this part.

At this point the central challenges and the technical lines of drone navigation have been presented in full. The next chapter widens the view from a single machine to a group: how do several robots map together? How are their separate maps fused? How do the cloud and the edge divide the labor? And what questions arise in communication, synchronization, and security? Chapter 27 completes the last piece of Part VII.

26.8 The State of Visual Localization on Drones: Where Is the Limit of Speed

This chapter has measured a great many places where things break. By way of closing, this section sets out the present boundary of the technology and three roads through it.

One: the Three Speed Bands of the Present State

Speed band	Typical setting	The state of visual localization	Chief limitation
Within 3 meters per second	Indoor inspection, filming	Mature; VI-SLAM runs steadily	Computation and endurance
3 to 10 meters per second	Outdoor inspection, survey	Workable, with the shutter and isolation to match	Motion blur

10 to 20 meters per second	Racing, fast search and rescue	At the research stage; needs stereo and good light	Feature tracks too short
Above 20 meters per second	The limit of racing	Vision can hardly carry it alone	Gone in a single frame

Table 26-5 Four Speed Bands of Visual Localization on a Drone

The dividing points of this table are exactly the two turnings Figure 26-2 measured: between 3 and 5 meters per second blur begins to bite, and at about 17 meters per second the feature track falls below three frames. The two limits are of different natures—the first can be relieved by optical means, the second is a hard geometric limit.

Two: Three Roads Through

The first is to change the sensor. An event camera outputs no frames but an event at each pixel whenever its brightness changes, with a time resolution down to microseconds, so that in principle motion blur does not exist. The price is that the form of its output is entirely different, and the whole apparatus of Chapters 20 to 23 must be rewritten.

The second is to let the IMU carry more. Section 26.4 noted IMU-based motion prediction improving feature tracking across frames, which is in essence using inertial information to predict where a feature ought to have moved to, narrowing the search from the whole image to a small window. At high speed this is of particular value—but it presumes that the IMU bias is estimated accurately, and Figure 26-3 has just shown how vibration destroys exactly that. The two hold each other in check, and this is the thorniest part of designing a drone system.

The third is to accept the limit and change the task. Since vision cannot hold up at high

speed, keep attitude at high speed on the IMU and the barometer alone, and slow down periodically to 2 or 3 meters per second for one visual correction of position. This is the same line of thought as the map maintenance of the inspection robot in Section 19.6 and the event-driven computation of Section 25.8—not to have the system meet the highest standard at every instant, but to have it meet the standard when it must.

For a student project the third road is the most practical. It needs no special hardware and no rewriting of the algorithm, only the writing of when to slow down into the plan of the task.

Formulas and Procedures

One: Converting Optical Flow to Speed

What a downward camera measures as optical flow is how many pixels the image moves per second; to convert it into a true horizontal speed one needs the height above the ground and the focal length of the camera:

$$v = (flow_pixels \,/\, f) \times h$$

where flow_pixels is the pixel displacement per second, f the focal length (in pixels, the intrinsics of Section 9.1), and h the height above the ground (supplied by the ranging sensor).

This formula reveals one crucial risk: the height h appears in the numerator. If the ranging sensor misjudges the height (if the height suddenly shrinks while flying over a tabletop, say), the converted speed is wrong, and the flight controller may make a wrong correction and drift or even collide. This is the commonest cause of failure in optical flow positioning, and students must mind the unevenness of the floor in their

experiments.

Two: Quantifying Motion Blur

The length of the blur on the image during the exposure is approximately:

$$blur_pixels \approx (v / h) \times f \times t_exposure$$

where v is the flight speed, h the distance to the scene, and t_exposure the exposure time.

Put in real numbers for the feel of it: at a flight speed of 5 m/s, a distance of 5 meters, a focal length of 400 pixels, and an exposure of 10 milliseconds, the blur is about 4 pixels—already enough to affect corner detection seriously (the features of Chapter 20 need clean local structure). Raise the speed to 20 m/s and the blur reaches 16 pixels, at which feature extraction fails almost entirely.

The formula also points directly to the remedy: shortening the exposure time t_exposure reduces the blur linearly—which is exactly the mathematical ground of the first remedy of Section 26.4.

Three: The State and the Error Terms of VIO

The state vector of a drone's VIO is the same as in Section 23.3, and for every key frame is:

$$S = \{ R, p, v, b_g, b_a \} \quad (15 \text{ degrees of freedom in all})$$

Tightly coupled optimization minimizes the visual and the inertial error together:

$$min \, \Sigma \rho(|| e_visual ||^2) + \Sigma || e_inertial ||^2$$

For a drone one practical point is the dynamic adjustment of the two weights: when blur or a shortage of features is detected, the weight of the visual term should be

lowered and that of the inertial term raised—which is exactly how the fifth remedy of Section 26.4 (adaptive weighting) is realized, and answers to the central idea of the Kalman gain in Chapter 11, that of weighting by credibility.

Four: A Layered Architecture for GPS-Denied Drone Navigation

Figure 26-4 The Layered Architecture of Drone Navigation and Its Graceful Degradation

【Correcting layer】at irregular intervals: UWB / visual markers / return to a GPS region → an absolute correction of position

↓ degrade on failure

【Upper layer】10–30 Hz: VI-SLAM / VIO → a full 6-DoF pose + a map

sustains: path planning, autonomous navigation, exploratory mapping

↓ degrade on failure

【Middle layer】tens of Hz: optical flow + ranging → horizontal velocity + height above the ground

sustains: holding a hover (at least not drifting away)

↓ degrade on failure

【Bottom layer】hundreds of Hz: IMU → attitude

sustains: keeping the flight steady so the operator may take over

【Design principle】let failure degrade step by step rather than collapse in an instant

Cases

Case 26-1 The Price of Having No Wheels: Why a Drone Is Harder Than a Ground Robot

A student who has done SLAM on both a ground robot and a drone will feel keenly how much harder the latter is. The reason is not merely that it flies faster, but a structural difference often overlooked: a drone has no odometry.

Recall the line of this book: Section 13.4 explained that 2D SLAM works in a cycle of prediction by odometry and correction by LiDAR; Section 14.5 noted that of Gmapping's three preconditions odometry is necessary; and the LOAM of Section 18.3 likewise needs a motion estimate as the initial value for registration. Odometry drifts, but it supplies a steady, continuous, free reference for motion.

A drone has no such luxury at all. Its only internal source of motion is the IMU, and Section 7.3 has already proved that a consumer-grade IMU integrated alone drifts by meters within seconds. This means that a drone's visual system must bear alone the responsibility odometry used to share—and once vision fails, the system has no reliable fallback.

This explains three things: why the drone field sets such store by the tight coupling of VIO (Chapter 23), why optical flow positioning is standard equipment on a consumer drone (Section 26.3), and why the layered degradation architecture of Section 26.2 is so crucial.

The lesson for students is this: understanding what a platform lacks often explains the trade-offs of its system design better than understanding what it has. This way of reasoning backward from constraints to design is the core of engineering judgment.

Case 26-2 The Dilemma of Initialization: Too Little Motion Will Not Do, and Neither Will Too Much

This is a contradiction of great teaching value, and one students are certain to meet in practice.

On the one hand, Section 23.4 stressed repeatedly that IMU initialization needs excitation of motion enough, without which scale and bias are unobservable. At rest or moving in a straight line at constant velocity, the system simply cannot initialize.

On the other hand, the instance mentioned in Section 26.4 shows that the machine hall sequences of EuRoC all begin with fast motion to assist IMU initialization, but that the motion blur this produces is a great challenge to visual tracking. Fast motion meets the need of the inertial system while spoiling the quality of the visual system's input.

That this contradiction exists shows exactly what visual-inertial fusion is in essence: the needs of the two sensors do not agree and even pull against each other. The IMU likes motion with acceleration in it; the camera likes a steady, clean picture.

The practical solution is a compromise: make a motion of moderate amplitude, with acceleration, deceleration, and rotation, but not too fast. Concretely, hold the device or fly the drone back and forth and side to side in small movements while turning for a few seconds, keeping the speed below the point at which the picture blurs noticeably.

This case also reminds students that when a method from a paper will not run in your hands, the trouble need not be in the program but may be that your data do not meet the method's implicit assumptions. Understanding what a method requires of its input is a precondition of implementing it successfully.

Case 26-3 The Boundary of Speed: How Fast Can VIO Fly

The answer to the question of how fast a flight visual localization on a drone can support has been pushed forward steadily in recent years, and displays the development of the technology clearly.

Take the baseline first: the EuRoC micro air vehicle dataset seldom exceeds 5 meters per second—which is the evaluation condition of most VIO papers. Against that baseline, most of the accuracy figures students see were obtained at relatively gentle speeds.

Pushing upward: research has supported up to 17.5 meters per second with a stereo RGB VIO, but it depends on stereo vision and presumes favorable lighting. A racing drone dataset may reach 28 meters per second, but usually with RGB cameras on a structured indoor course or a short outdoor track.

Breaking the limit: more advanced research has begun to use unconventional sensors. One work achieved real-time operation at 30 meters per second in low-altitude flight with a monocular thermal-inertial odometry. The advantage of thermal imagery is that it is unaffected by light and is less sensitive to certain sources of blur.

But this road has its price too. Research points out that although methods based on deep learning show good gains in performance on thermal imagery, their inference time remains incompatible with the strict limits of latency and power on a drone's onboard hardware—certain methods need more than 30 milliseconds per frame even on a laptop-class GPU. A hand-designed pipeline therefore remains the most practical solution for embedded high-speed closed-loop flight.

This case distills a theme that appears again and again in this book: the splendid

figures of an advanced technology often rest on particular conditions, while a system that can really be deployed must satisfy accuracy, speed, power, and reliability all at once. Learning to read a paper for what was achieved and under what conditions at the same time is an important part of the ability to do research.

Implementation Activities

Implementation 26-1 A Drone Visual-Inertial Navigation Experiment

Purpose: to understand the challenges of visual localization on a drone and what can be done about them, through a public dataset and (optionally) real flight. Parts One and Two need no hardware and suit everyone; Part Three needs a micro drone platform.

Part One: Analyzing a Dataset (everyone; no hardware needed)

1. Download the EuRoC dataset, choose the two sequences MH_01_easy and V1_03_difficult, run each in the stereo-inertial mode of ORB-SLAM3 from Chapter 23, and compute the ATE with evo.
2. Compare the difference in accuracy between the two, and play the images to see in which passages of the difficult sequence motion blur appears plainly, setting this against the discussion of Section 26.4.
3. Quantify the blur: write a program to compute the sharpness of every frame (the variance of the Laplacian will serve), draw the curve of sharpness against time, and mark the blurriest passages.
4. An analysis of correlation: draw the sharpness curve together with the curve of trajectory error and see whether the two are related—is the moment of blurriest image also the moment of greatest error?

5. Observe the initialization: watch the fast motion at the start of the sequence, record the time the system needs to complete IMU initialization, observe how blurred the images are over that stretch, and verify the dilemma of Case 26-2 for yourself.

Part Two: Simulating Optical Flow and Blur (everyone)

1. Using OpenCV's Lucas-Kanade optical flow, compute the flow on a downward-looking video (a phone moved horizontally above a tabletop will serve) and convert it into a horizontal speed by Formula One.
2. A sensitivity experiment on height: set the height wrongly on purpose (enter 0.5 meters where it is really 1 meter), observe the error in the converted speed, and understand the risk of failure mentioned in Section 26.5.
3. Simulate blur: add motion blur artificially to a clean image (with OpenCV's motion blur kernel), the blur length computed by Formula Two, and test how the number of ORB features detected falls as the blur grows.
4. Draw the curve of blur length in pixels against the number of features detectable, and find the critical point at which the number of features falls away sharply. That critical value is the ground of the upper speed limit your system can tolerate.

Part Three: Real Flight (for classes with a micro drone platform)

Read the safety principles of Section 26.5 first without fail, and work inside a net with a micro platform.

1. Test the hover: start optical flow positioning, let the drone hover in place for 30 seconds, and measure the extent of the horizontal drift against markings on the floor.

2. An experiment on ground texture: hover in turn over a patterned carpet, a plain floor, and a reflective floor, compare the extent of the drift, and verify optical flow's dependence on texture.
3. An experiment on height: hover at several heights above the ground (0.5, 1.0, 1.5 meters), observe the difference in steadiness, and think about the effect of an error in the height in Formula One.
4. A simple route: set a square route to be flown autonomously, measure the discrepancy on returning to the start, and assess the accumulated drift.
5. A challenge test: raise the flight speed step by step, safety permitting, record the speed at which the system begins to be unsteady, and set this against the discussion of Case 26-3.

Discussion Questions

1. By what factor do the ATEs of the easy and the difficult sequences differ? How well are image sharpness and trajectory error correlated?
2. By Formula Two, if your camera's exposure is 5 milliseconds, its focal length 400 pixels, and the scene 3 meters away, at what flight speed does the blur reach 5 pixels? What does this imply for an upper speed limit?
3. In the optical flow conversion experiment, how much error in speed does a 50 percent error in height produce? If the drone corrects on that wrong speed, what will happen?
4. (For those who fly) Over which floor was the hover steadiest? What made the least steady one least steady? Which limitation of Section 26.3 does this bear out?

5. By the layered architecture of Figure 26-1, if your drone's VIO fails suddenly in flight, how should the system respond? Design a procedure for degrading gracefully.

Extended Challenge (university level)

First: implement adaptive weighting—add an assessment of image quality to a VIO system (the sharpness index of Part Two, say), lower the weight of the visual term automatically and raise that of the inertial term when the image is blurred, and test on the EuRoC difficult sequences whether the accuracy improves. This corresponds to the fifth remedy of Section 26.4 and is a complete small research topic in itself. Second: a dataset of a university site—by the method of Section 26.5, record visual-inertial data inside a departmental building with a micro drone or a handheld sensor rig (images, IMU, and an approximate ground truth from AprilTags), put it into a standard format, and publish it for the students who come after. This is a contribution that yields value over a long time, and a fine form in which to present the result of a project.

Further Reading

1. M. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," International Journal of Robotics Research, 2016. (The official description of the EuRoC dataset, the basis of Section 26.6.)
2. D. Schubert et al., "The TUM VI Benchmark for Evaluating Visual-Inertial Odometry," IROS, 2018. (The TUM-VI dataset, with an account of its photometric calibration and hardware time synchronization.)
3. C. Campos et al., "ORB-SLAM3," IEEE T-RO, 2021. (The system of Chapter 23, and the principal tool for this chapter's implementation.)

4. T. Qin, P. Li, and S. Shen, "VINS-Mono: A Robust and Versatile Monocular Visual-Inertial State Estimator," IEEE T-RO, 2018. (Another representative work in drone visual-inertial systems.)
5. D. Scaramuzza and Z. Zhang, "Visual-Inertial Odometry of Aerial Robots," Encyclopedia of Robotics, Springer, 2019. (The authoritative survey of visual-inertial navigation for aerial robots, from the UZH group, answering to Chapter 2.)
6. K. Sun et al., "Robust Stereo Visual Inertial Odometry for Fast Autonomous Flight," IEEE RA-L, 2018. (A representative work on VIO in fast flight, mentioned in Case 26-3.)
7. The documentation on integrating visual localization with the PX4 and ArduPilot open-source flight controllers. (Practical guidance on feeding VIO output into a flight controller; essential reading before real flight.)

Summary

1. Localizing a drone has four particular difficulties: full six-degree-of-freedom motion, no odometry (only a drifting IMU), high speed and violent dynamics, and severe consequences of an error; together with the constraints of payload and power and the severe demand on timeliness, these make it one of the hardest SLAM platforms of all.
2. GPS-denied settings include indoors, beneath bridges, tunnels, urban canyons, under vegetation, and deliberate interference; the means available, in order, are the IMU, the barometer, optical flow, ranging, VIO, VI-SLAM, and UWB or visual markers, and practice adopts a layered architecture so that failure degrades step by step rather than collapsing in an instant.

3. The levels of the three visual localization technologies: optical flow gives only horizontal velocity but costs very little computation and sustains a hover in place; VIO gives a full 6-DoF pose but drifts; VI-SLAM adds a map and loop closure and corrects long-term drift. One should choose by what the task requires and use no technology beyond it.
4. Fast flight deals a triple blow: motion blur damages feature tracking, shortened feature tracks make mid-term association hard to establish, and latency in the control loop worsens; the remedies include raising the shutter speed, using a global shutter, improving tracking with IMU prediction, enhancing the image, and adjusting the weights of vision and inertia adaptively.
5. The safety principles of an indoor flight experiment are not to be compromised: a net, a micro platform, an emergency stop, and testing in stages; the site needs attention to texture, light, and air currents; and the procedure should advance from a hover test by degrees to a challenge test.
6. EuRoC supplies eleven micro air vehicle sequences with stereo images and motion capture ground truth, at speeds seldom above 5 meters per second; TUM-VI supplies 1024 × 1024 images with photometric calibration and hardware time synchronization, with a greater variety of scenes. In comparing figures one must mind the evaluation conditions of each study.
7. Drone autonomy may be divided into three levels—stable, autonomous, and intelligent; the technology advances along three main lines, computation settling downward, sensing growing lighter, and intelligence rising, and the three are converging: a drone that is small, frugal, and capable of semantic understanding is becoming a reality.

Carrying Forward

This chapter has quantified the three new problems flight brings. On drift: over the second half of a 72-second flight the mean error is 0.810 meters for optical flow, 0.296 for VIO, and 0.060 for VI-SLAM, and the real difference lies in the shape—only VI-SLAM's error does not grow with time. On speed: the feature matching rate falls from 100 percent at 2 meters per second to 21 percent at 28, and more deadly still the number of frames a feature stays in view falls from 104 to 2; the most effective remedy is the shutter (at 17 m/s, going from 1/125 to 1/500 raises the matching rate from 11 percent to 80). On vibration: aliasing disguises the propellers' high-frequency vibration as a false bias of 1.00 meters per second squared (which produces an 8-meter position error in 4 seconds), and raising the sampling rate does nothing—only mechanical isolation before sampling presses it down to 0.014.

This chapter has completed the discussion of navigation for a single drone. But however advanced it is, the field of view, the endurance, and the computing power of one robot are limited in the end. The next chapter widens the view to the group: how do several robots map together? How are their separate maps fused into one consistent global map? How do the cloud and the edge divide the labor? And what questions arise in communication bandwidth, time synchronization, and data security? Chapter 27 completes the last piece of Part VII and prepares the ground for the implementation of Part VIII and the AI semantic navigation of Part IX.

Taiwan-Authored University Textbook

@prof

SLAM

Simultaneous Localization and Mapping, SLAM

Chapter 27

Multi-Robot SLAM and Cloud / Edge Collaboration

Editor: Lee Shihyuan (李世淵) LINE OA @prof

Assistant: Aaron Lee, Anne Lee

University Intelligent Robotics Program / High School AI and Robotics Club Course

Where This Chapter Sits in the Book

Chapter 25 made the robot small and Chapter 26 set it flying. The shared conclusion of the two chapters is that the ability of a single platform is limited in the end. This chapter's answer is not to make it stronger but to bring more of them.

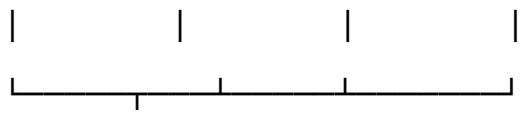
But bringing more of them is not multiplying the ability of one by N. A multi-robot system has problems of its own—how the maps are aligned, who talks to whom, what is to be sent first when the bandwidth will not stretch, and whether one robot mistaking its place will bring all the others down with it. None of these exists in a single-robot system.

The three figures of this chapter quantify three of these things: Figure 27-1 computes how much faster several robots exploring together really are, and what sharing a map is worth; Figure 27-2 measures how much damage one wrong cross-robot loop closure can do; and Figure 27-3 answers a very practical question—when the bandwidth is enough for only a few loop closures, which ones should be sent.

Figure 27-0 The Link Map of Chapter 27

The foundations built up so far

Chapter 12 Pose graph optimization Chapter 22 Loop closure and covisibility Chapter 23 Merging multiple maps Chapters 25–26 Constrained platforms



▼

【Chapter 27 If one will not do, use a group】

Map fusion ─ three architectures ─ the cloud-edge division of labor ─ communication and

security

▼

Part VIII ROS implementation (making all of this run)

▼

Part IX AI semantic navigation (the case in Chapter 34: collaborative exploration by several machines)

Learning Objectives

The twenty-five chapters so far have discussed one robot. But the field of view, the endurance, and the computing power of a single machine are limited in the end: one robot dog needs hours to patrol a whole departmental building, where ten might need only twenty minutes. This chapter widens the view to the group and completes the last piece of Part VII. After studying this chapter, students should be able to:

1. Explain the advantages of multi-robot SLAM over a single machine and the three great challenges it adds.
2. Explain the procedure of map fusion and what is crucial in detecting a cross-robot loop closure.
3. Compare the merits and defects of the centralized, distributed, and decentralized architectures of collaboration.
4. Explain the principles by which cloud SLAM and edge SLAM divide the labor, and where each is suitable.
5. Plan the system architecture and the allocation of tasks for a campus inspection by several robot dogs.
6. Explain how SLAM and an AI agent work together, and the role of a VLM in coordinating several machines.

7. Explain the questions a multi-robot system raises in formation, communication, data synchronization, and security.

Core Concepts

中文名詞	English Term	Explanation
協作式 SLAM	Collaborative SLAM	The technology by which several robots build one consistent map together.
集中式架構	Centralized	All the data are gathered on a central server to be processed.
分散式架構	Distributed	The computation is spread over several nodes, which exchange information.
去中心化	Decentralized	No central node; the robots communicate directly and each fuses for itself.
地圖融合	Map Fusion / Merging	Aligning several maps and merging them into one consistent global map.
跨機器人閉環	Inter-robot Loop Closure	A constraint established where different robots have observed the same place.
相對變換	Relative Transform	The rotation and translation between two robots' coordinate frames.
會合	Rendezvous	The event of two robots meeting and exchanging information.
感知混淆	Perceptual Aliasing	A wrong association caused by different places looking the same.
離群點剔除	Outlier Rejection	Excluding wrong loop closures so that they do not spoil the global map.
邊緣運算	Edge Computing	Computing on a local node close to the robot.
雲端運算	Cloud Computing	Computing and storing in a remote data

中文名詞	English Term	Explanation
		center.
異質團隊	Heterogeneous Team	A team made up of robots of different kinds (ground, aerial).

27.1 Single-Robot SLAM and Multi-Robot SLAM

This section explains why several robots are used, and what new problems the choice brings.

Four Advantages of Several Robots

1. Speed: several machines explore separately and the time to finish a large task falls greatly. This is the most direct motive—the collaborative multi-robot exploration of Case 24-3 in Chapter 24 is exactly this line of thought.
2. Fault tolerance: the failure of one does not affect the task as a whole. By contrast, when a single expensive robot fails the whole task stops.
3. Complementary viewpoints: observing from several positions at once allows a more complete model of the environment and reduces the blind spots occlusion causes.
4. Complementary capabilities: a heterogeneous team can combine the strengths of different platforms—a ground robot has long endurance and a large payload, a drone a wide view and the ability to go high (the ground-air collaboration mentioned in Section 26.7).

Three Great New Challenges

But several robots are not simply one multiplied by N; three problems arise that do not exist for a single machine:

- Unifying the coordinate frames: every robot takes its own starting point as its origin when it is switched on (Chapter 10). To put their maps together one must first know the relative transformation between their coordinate frames—and that transformation is unknown at the outset.

- The widening of data association: single-robot SLAM need only judge whether what I now see is something I have seen before (the long-term association of Chapter 23); several robots must further judge whether what I see is something someone else has seen. The search space grows and with it the risk of a wrong association.
- The limits of communication: the bandwidth between robots is limited, the link may drop, and the latency is not fixed. Every mechanism of collaboration must work within that reality—the theme that recurs throughout this chapter.

Its Relation to the Single-Machine Technology

The good news is that students have already learned the core technologies of multi-robot SLAM.

- Each robot localizing and mapping for itself: the various single-machine SLAMs of Chapters 13 to 26.
- Detecting cross-robot loop closures: the place recognition (the bag-of-words model) and geometric verification of Chapters 20 and 22.
- The optimization of map fusion: the pose graph optimization of Chapter 12.
- Handling wrong loop closures: the robust kernels and outlier rejection of Chapters 12 and 15.

In other words, multi-robot SLAM extends the single-machine technology to several coordinate frames and several sources of data. Understand this and students will not find the content of this chapter strange—it is the natural extension of all the chapters before.

Measuring It: How Much Faster Are a Few More Machines

Section 27.1 listed four advantages of multi-robot SLAM, the first being speed. It sounds the most self-evident of them and is also the most easily overestimated. This subsection computes it.

The experimental setting is a departmental floor of 34 × 18 meters (a central corridor with twelve rooms on either side). N robots set out from different positions along the corridor and each explores by the strategy of going to the nearest frontier, with a sensing radius of 3 meters, until 98.5 percent of the free space has been explored. Two settings are compared: sharing a single map (so that each can see where the others have been) and mapping separately, with no exchange of information at all.

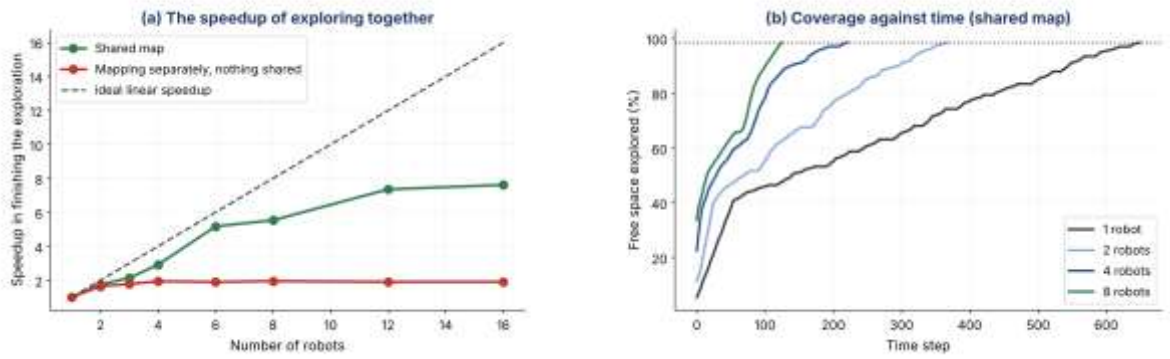


Figure 27-1 The Speedup of Collaborative Exploration by Several Robots

Panel (a) is the speedup. With a shared map two machines give 1.75 times, three 2.14, four 2.91, six 5.17 and eight 5.54—after which it flattens: twelve give 7.35 and sixteen 7.61. That is to say, the ceiling of parallelism on this floor is about twelve machines. More robots than that are not useless; it is that the time they need to travel to the last few unexplored corners has become a bottleneck that cannot be compressed further.

Note also that the speedup never reaches the dashed line of ideal linear scaling, and falls further below it the more machines there are: four give 2.91 rather than 4, and sixteen 7.61 rather than 16. Part of what is lost is travel—the robots must still walk to

the frontier—and part is overlap, where two of them approach the same unexplored corner from opposite sides.

What really deserves attention is the red line. With no map shared, two machines give only 1.64 times, four 1.94, and the curve then stops dead—eight give 1.95 and sixteen 1.91, however many are added. That flat line is itself the conclusion: robots that do not share a map explore again the regions the others have already covered, so that beyond two of them the extra machines add almost nothing. For eight machines, sharing the map shortens the time to completion by 82 percent.

This is why the map fusion of Section 27.2 is the central operation of the whole chapter. Without fusion, several robots are merely several single machines each going its own way; with fusion they truly become one system.

Once the Speedup Saturates, What Do the Extra Robots Do

Figure 27-1(a) shows that beyond six machines it grows no faster. But this does not mean the seventh is of no value.

The other three advantages mentioned in Section 27.1—fault tolerance, variety of viewpoint, and the division of labor—do not saturate with number. The thirteenth machine can take over when one fails, can observe the same object from another angle to make a semantic judgment more reliable (Chapter 32), or can be dedicated to serving as a communication relay.

In planning the size of a fleet one should therefore ask two questions separately: how fast must it be finished decides the number needed for the exploration, and how reliable must it be and what else must it do decide the number beyond that.

27.2 Map Fusion

Map fusion is the central operation of multi-robot SLAM: aligning two (or more) separately built maps and merging them into one consistent global map.

What It Shares with and What Distinguishes It from Single-Machine Loop Closure

Students should think at once of the Atlas map merging of Section 23.6. The mathematical essence of the two is the same—find the relative transformation between two maps, then propagate the correction by pose graph optimization. The difference is this:

- Several maps on one machine: the two maps come from different periods of the same robot, with exactly the same sensors and parameters.
- Map fusion across robots: the two maps come from different robots and may use different sensors, different viewpoints, and even different map representations. Fusion in a heterogeneous team is harder still.

The Three Steps of Fusion

1. Detect the encounter: judge whether two robots have observed the same place. This may be a physical meeting (the two robots see each other) or an overlap in place recognition—the scene robot A sees now, robot B saw a little earlier.
2. Estimate the relative transformation: once the overlap is confirmed, solve for the rotation and translation between the two robots' coordinate frames from the features or scans they observed in common (the geometric solution of Chapter 21, or the point cloud registration of Chapter 17).

3. Fuse and optimize: join the two pose graphs with this cross-robot edge and run pose graph optimization (Chapter 12) so that the whole system becomes globally consistent.

The Crucial Risk: a Wrong Cross-Robot Loop Closure

Chapters 12, 15, and 22 all stressed that one wrong loop closure drags the map bodily to the wrong place. In a multi-robot system that risk is higher still, because:

- The space of candidates is larger: what must be compared is not only one's own history but the observations of every other robot.
- Perceptual aliasing is worse: different robots may be at different but similar places (two classrooms of identical layout, say) and produce a wrong match that looks right.
- The consequences are more serious: one wrong cross-robot edge spoils the maps of several robots at once.

Robust outlier rejection is therefore an indispensable component of multi-robot SLAM. Systems in this line integrate robust outlier rejection to strengthen the practice and to adapt it to sporadic communication between robots. Another system adopts a distributed graduated non-convexity algorithm and incorporates semantic data into the map it produces.

Considerations of Communication Efficiency

Map fusion requires an exchange of data, and bandwidth is limited. How to achieve the best fusion with the least data is therefore the central question of the field. One system introduces a novel technique for prioritizing cross-robot loop closures by maximizing algebraic connectivity, so as to reduce communication and speed

convergence; it also adopts decentralized neighbor management and pose graph optimization suited to sporadic inter-robot communication, and supports descriptors for different sensor types—CosPlace and NetVLAD for images, Scan Context for LiDAR scans.

The central insight here is that not all loop closure candidates are of equal importance. Some contribute a great deal to global consistency and others scarcely matter. Transmitting the high-value candidates first obtains the greatest benefit within a limited bandwidth—a line of thought full of engineering wisdom.

Measuring It: How Dangerous Is One Wrong Cross-Robot Edge

Section 27.2 says that one wrong cross-robot edge spoils the maps of several robots at once. The sentence deserves to be said once in numbers, because its severity exceeds most people's intuition.

The experimental setting: three robots each walk a partly overlapping circular route of 50 key frames, with noisy odometry. Between them there are 42 correct cross-robot loop closures. A number of wrong edges from perceptual aliasing are then mixed in deliberately—the robot believes it is seeing the other's frame j when in fact it is another place that merely looks much like it. Ordinary least squares and a robust kernel (graduated non-convexity) are compared as ways of solving.

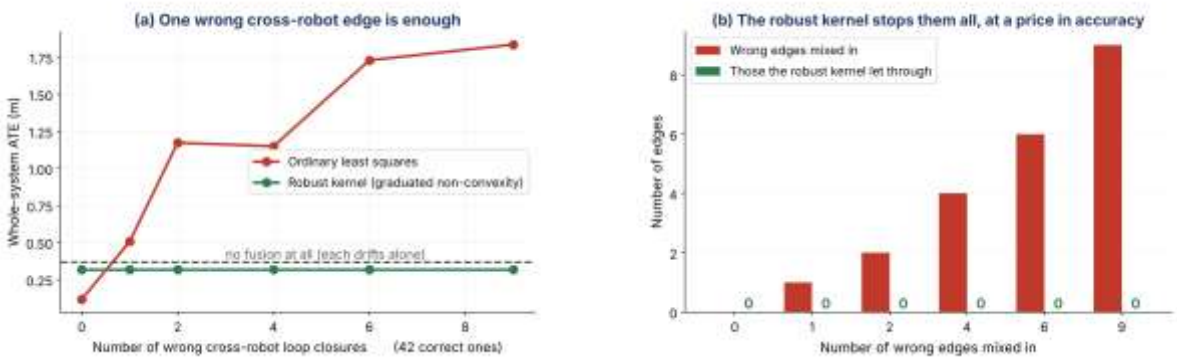


Figure 27-2 Wrong Cross-Robot Loop Closures and Robust Rejection

The red line of panel (a) is the point. With no wrong edges, the whole system's ATE after fusion is 0.118 meters, more than three times better than the 0.369 meters of no fusion at all—this is the positive value of map fusion. But let a single wrong edge in and the ATE leaps to 0.507 meters; two give 1.17 meters; and nine give 1.84.

Note what one wrong edge does: from 0.118 up to 0.507, more than fourfold, and worse than not fusing at all. In other words, in this situation fusing with one wrong edge mixed in is worse than not fusing in the first place. That is why Section 27.2 lists outlier rejection as an indispensable component—it is not there to raise accuracy but to avert disaster.

The green line is the result with the robust kernel added. However many wrong edges are mixed in (up to nine were tested), the ATE holds steady at 0.319 meters and does not worsen at all as the wrong edges multiply. Panel (b) shows its power of discrimination: every wrong edge was stopped, and not one slipped through.

But there is a price here that must be stated honestly. Even with no wrong edges at all, the robust kernel's ATE is 0.319 meters against ordinary least squares' 0.118—nearly three times worse. The reason is that a robust kernel lowers the weight of all large residuals, including correct edges that merely happen to be noisier. It has traded

accuracy for immunity to outliers.

The practice is to work in two stages: use the robust kernel first to identify the inlier set, then redo an ordinary least squares on the inliers alone. In theory this obtains the benefits of both, but the threshold of discrimination must be set with great care—loosen it and a wrong edge gets in, and one is enough to undo everything. This is the part of multi-robot SLAM that most needs tuning in practice and most easily goes wrong.

Measuring It: When the Bandwidth Will Not Stretch, Which Ones Should Be Sent

Section 27.2 ended by noting that not all loop closure candidates are of equal importance, and introduced the practice of ordering them by maximizing algebraic connectivity. The idea is rather abstract, and this subsection makes it concrete by experiment.

Algebraic connectivity is the second smallest eigenvalue of a graph's Laplacian matrix, and measures intuitively how tightly the graph hangs together—the larger it is, the harder the graph is to cut in two and the more easily error is redistributed across the whole of it (the reasoning of Figure 22-2 in Chapter 22). The experiment compares two strategies of selection: picking k candidates at random, and greedily picking each time the one that raises the algebraic connectivity most.

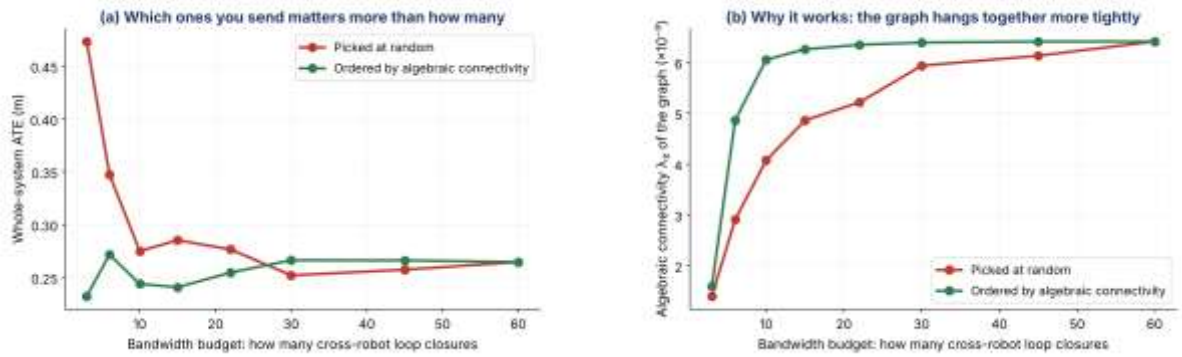


Figure 27-3 Under a Limited Bandwidth, Which Loop Closure Candidates Should Be Sent First

Panel (a) is the result. With a budget of only 3, random selection gives an ATE of 0.473 meters and ordering by connectivity 0.232 meters; with a budget of 10 the figures are 0.275 and 0.244 meters; with a budget of 22 they are 0.277 and 0.255. At a small budget the ordering halves the error, and at a middling one it still holds a clear lead.

A more useful way to read it is horizontally: ordering by connectivity needs only 10 to reach 0.244 meters, where random selection needs 30 to reach a comparable level. That is to say, simply choosing which ones to send can bring the bandwidth requirement down to a third. For a team of robots with only sporadic chances to communicate, this difference is decisive.

Panel (b) explains why it works. The vertical axis is the algebraic connectivity of the graph, and the green line is above the red at every budget—the ordering strategy has bought a more tightly connected graph with the same number of edges. The two lines meet at a budget of 60 (everything sent), because there the two are choosing the same set.

This experiment also demonstrates a general line of engineering thought: when resources are limited, rather than asking whether a few more can be had, ask first

whether those on hand are being spent where they count. The same thought appeared in the key frame culling of Chapter 22 and the event-driven computation of Chapter 25.

Why Algebraic Connectivity Is the Right Index

Intuitively one would want to pick the most certain loop closure, or the nearest. But such loop closures often connect two nodes that are already close together—the information is redundant and helps global consistency little.

Algebraic connectivity catches something else: whether this edge joins two pieces of the graph that were far apart. Stitching two loosely connected regions together is far more useful than putting another stitch where it is already tight.

This is the same reasoning as Chapter 22's covisibility edges carry the correction along the whole trajectory, only turned from a qualitative observation into a number that can be computed and ordered by.

27.3 Three Architectures for Collaborative SLAM

The system architecture of multi-robot SLAM is the most important design decision of this chapter. The spectrum of multi-robot SLAM architectures runs from the centralized all the way to the fully decentralized.

One: the Centralized Architecture

All robots send their data to a central server, which processes and optimizes them together and distributes the results back.

Centralized systems of the CCM-SLAM kind gather the data on a server for joint optimization; this simplifies map fusion but introduces a communication bottleneck

and a single point of failure.

- Merits: map fusion is simple (all the data are in one place), global optimization is easy, and the computational burden on the robots is light.
- Defects: a communication bottleneck (everything must be transmitted), a single point of failure (the server goes down and the whole system stops), and poor scalability.
- Representative systems: CCM-SLAM, COVINS (for visual-inertial sensing, sparsifying by removing redundant key frames), and LAMP 2.0 (a LiDAR system that predicts the priority of every cross-robot loop closure candidate with a graph neural network).

Two: the Distributed and Decentralized Architectures

In a distributed system the processing is spread over several nodes or agents; the robots collect data and carry out their own localization and mapping while sharing information with one another through continuous communication, so as to build a complete global map.

In the decentralized practice the robots exchange information with one another directly and complete all data fusion onboard, depending on no central entity to coordinate the system.

- Merits: no single point of failure, good scalability, and greater resilience to a break in communication.
- Defects: high complexity of synchronization, a need for distributed optimization algorithms, and a heavier computational burden on each robot.

- Representative systems: DOOR-SLAM and Kimera-Multi perform peer-to-peer pose graph optimization with outlier rejection, trading higher complexity of synchronization for greater scalability; Swarm-SLAM offers a decentralized ROS 2 framework supporting several sensing modalities; and LAMP and LAMP 2.0 handled underground multi-robot SLAM in the DARPA Subterranean Challenge (Case 1-2 in Chapter 1).

What Decides the Architecture: the Conditions of Communication

The defect of a centralized architecture is that it requires good communication with a common back end, and that if that node fails the whole system fails; moreover it cannot scale to a large team of robots.

A decentralized solution depending only on sporadic, intermittent communication between robots therefore suits a large setting better.

Architecture	Where the computing is	Merits	Defects	Where it suits
Centralized	A central server	Fusion is simple; the robots are light	A communication bottleneck, a single point of failure	A small team with good communication
Distributed	Several nodes working together	No single point of failure; scales well	Synchronization is complex	A medium or large team
Decentralized	Onboard each robot	Resilient to a break in	A heavy burden of onboard	A large-scale task with

Architecture	Where the computing is	Merits	Defects	Where it suits
		communication	computation	sporadic communication

Table 27-1 A Comparison of the Three Collaborative SLAM Architectures

Hybrids in Practice

A real system is often not purely one or the other. For instance: the robots run their own SLAM onboard (the decentralized part) but send key frames to a server for global optimization when communication is good (the centralized part). Such a hybrid design takes the merits of both, and is the engineering way of thinking this book stresses repeatedly—there is no need to choose one of two textbook categories.

27.4 Cloud SLAM and Edge SLAM

The last section discussed how robots divide the labor among themselves; this one discusses how a robot divides the labor with computing resources outside it.

The Division of Labor Among Three Tiers

Tier	Where	Latency	Computing power	Tasks it suits
Onboard	The robot itself	Very low	Constrained (Chapter 25)	Real-time control, attitude estimation, obstacle avoidance
Edge	A local server / base station	Low (a few to tens of milliseconds)	Moderate to high	The SLAM back end, map fusion, AI inference

Tier	Where	Latency	Computing power	Tasks it suits
Cloud	A remote data center	High (tens to hundreds of milliseconds)	Very high	Large-scale map storage, model training, cross-site analysis

Table 27-2 The Division of Labor Among the Onboard, Edge, and Cloud Tiers

The Basic Principles of the Division

Three criteria decide what should be done where:

1. The demand on timeliness: the control loop must be onboard (the drone case of Section 26.1 is the plainest of all); only a task that can tolerate tens of milliseconds of latency is fit to be sent to the edge.
2. The volume of data: transmitting large volumes of raw data is costly. Compressing or extracting features onboard first and transmitting only the condensed result is far more efficient (which is exactly the line of thought of Section 27.2 on communication efficiency).
3. Security and privacy: is image data bearing on personal privacy fit to be sent to the cloud? In settings such as a hospital or a school this must be considered.

Edge-Assisted Collaborative SLAM

Edge computing suits multi-robot SLAM particularly well, because it solves two problems at once: the shortage of computation on the robot and the latency of a centralized architecture.

One system optimizes the server side by offloading the heavy computation while maintaining the agents' autonomy with onboard visual odometry. This is a typical

division: the robot does the real-time front end itself (depending on no network) and leaves the time-consuming back-end optimization and map fusion to an edge server.

Advances in communication technology are pushing this direction too. Research has used 5G communication to handle real-time dense volumetric data from several robots, showing how high throughput and low latency make it possible to offload large sensor streams. Section 5.4 mentioned 5G in robot localization; here is its value in collaborative computing.

Communication Quality as a Planning Resource

An important change of idea in recent years deserves students' attention. The central idea of one study is to treat communication quality as a planning resource and, combined with multi-agent collaboration, to make the navigation cliff a manageable event of degradation rather than a hard limit of operation.

This line of thought is most suggestive: rather than accept passively that there is no signal here, bring the quality of communication into path planning actively—have one robot stop where the signal is good to serve as a relay, say, or arrange for robots to return periodically to a communicating region to exchange data. It is of a piece with the design philosophy of Section 26.2, letting failure degrade step by step.

27.5 Campus Inspection by Several Robot Dogs

This section integrates the technology above into one concrete system plan, which is also the foundation of Project Four in Chapter 31.

Why Several Are Needed

Estimating by the scale of an ordinary university campus: one robot dog inspecting

at half a meter per second may need several hours to walk every corridor and floor of a departmental building, while its battery often lasts less than an hour. Only with several robot dogs working separately can the task be finished on a single charge.

The System Architecture

Figure 27-4 The System Architecture of Campus Inspection by Several Robot Dogs

【Onboard layer】on every robot dog

• LiDAR-inertial SLAM (Chapter 18) or visual-inertial SLAM (Chapter 23)

• Real-time obstacle avoidance and gait control

• Semantic perception: YOLO detection of inspection targets (Chapter 24)

↓ sent: key frame descriptors, the pose graph, inspection findings

【Edge layer】a local server on campus

• Cross-dog loop closure detection and map fusion (Section 27.2)

• Global pose graph optimization (Chapter 12)

• Task allocation and route coordination

↓ sent: the condensed map, the inspection report

【Cloud layer】(optional)

• Long-term map storage and comparison across time (change detection)

• AI model training and updating

• The management interface and reports

Strategies for Allocating the Task

- By area: the simplest—each robot dog takes a floor or a region. The merits are that they do not interfere with one another and that it is easy to implement; the defect is that it cannot be adjusted dynamically, so that a dog whose region finishes early stands idle.
- By frontier, allocated dynamically: the practice mentioned in Case 24-3 of Chapter 24—allocate the boundary between the explored and the unexplored (the frontier) dynamically to the most suitable robot. It is more efficient, but needs a mechanism of real-time coordination.
- By capability: the practice of a heterogeneous team—the robot dogs take the stairs and the interiors, the drones the atrium and the roof, and the wheeled robots the flat long corridors (where their endurance is best).

Practical Questions

1. Aligning the initial coordinates: let all the robot dogs set out from the same charging station, or place an AprilTag at the starting point (Section 24.3), and a common coordinate frame is established directly, which simplifies map fusion greatly. This is a fine example of avoiding a hard problem by a simple method.
2. Communication coverage: a campus Wi-Fi does not necessarily reach everywhere (basements, stairwells). The system must tolerate a dropped link and send the missing data when it recovers—which is exactly the value of the decentralized architecture of Section 27.3.
3. Safety and giving way: several robot dogs may meet in a corridor and must coordinate their passage. Interaction with pedestrians must also be considered (the socially aware navigation of Chapter 24).

4. Battery management: a robot dog's endurance is short, and the system must plan a charging rotation—two inspecting and one charging, say, taking turns to maintain continuous coverage.

27.6 SLAM Working Together with an AI Agent

Section 24.5 introduced VLM-assisted navigation; this section focuses on the role of AI in coordinating several robots.

The VLM as a Global Planner

Case 24-3 in Chapter 24 set out this direction in detail: the VLM does no low-level control but plays the commander—looking at the global map, reasoning from common sense about which directions are most likely to find the target, and allocating the tasks to the several robots.

The elegance of this division is that the VLM's strength is semantic understanding and common-sense reasoning (slow but clever) while the robot's is real-time perception and control (fast but without understanding of meaning). Letting each do what it is good at is the most reasonable architecture.

Three Roles for an AI Agent in a Multi-Machine System

1. The planner of tasks: decomposing a high-level goal (inspect every classroom on the third floor) into concrete subtasks and allocating them to the robots. This requires an understanding of the semantic structure of the site (the semantic map, Chapters 3 and 24).
2. The reader of anomalies: an inspection robot returns a great many images, and examining them one by one by hand is impractical. An AI agent can filter out

the anomalies that need human attention (a crack, a leak, a blocked passage) and lighten the burden of labor greatly.

3. The arbiter of coordination: when the needs of several robots conflict (all want the same region, all need charging), the agent decides by the global goal.

Sharing the Semantic Map

One advanced direction in multi-robot collaboration is to share semantics and not only geometry. One system incorporates semantic data into the map it produces, and can build a globally consistent metric-semantic three-dimensional mesh model in real time while depending on local communication alone, with resilience to wrong loop closures.

The value of sharing semantics is this: once robot A has recognized this is an elevator, robot B need not recognize it again but may use the result directly. This not only saves computation but keeps the whole team's understanding of the environment consistent.

A Practical Caution

The limitations Section 24.5 stressed apply in a multi-robot system too, and are even more severe:

- Latency: VLM inference takes time, and if every decision must wait on it the system reacts too slowly. AI should therefore handle decisions at the scale of seconds and above, while real-time avoidance remains the business of traditional methods.

- Hallucination: a wrong judgment by the AI may send the whole team exploring in the wrong direction. The system should retain verification at the geometric level and an interface for human supervision.
- Communication: an AI decision is usually made at the edge or in the cloud, which means it depends on communication. When the link drops the robots must be able to fall back on a default autonomous behavior (the degradation thinking of Section 27.4).

27.7 Formation, Communication, Data Synchronization, and Security

This section gathers the four engineering questions of a multi-robot system; unglamorous beside the algorithms, they often decide whether the system can really work at all.

One: Formation and Spatial Coordination

- Avoiding collisions: several robots moving in the same space must avoid striking one another. The practices include sharing positions in real time, dividing regions of travel, and setting rules of passage at narrow places (corridors, doorways).
- Maintaining the communication chain: if the team must stay connected, the formation must take account of communication range. Sometimes one robot is stationed deliberately at a relay position (treating communication as a planning resource, as Section 27.4 noted).
- Efficiency of coverage: the formation also affects the efficiency of exploration. Spreading out speeds coverage, but spreading too far loses the ability to support one another and to communicate.

Two: Communication

- Limited bandwidth: raw images and point clouds are enormous in volume and cannot all be transmitted. What is transmitted in practice is a condensed representation—key frame descriptors, the pose graph, sparse features (the considerations of efficiency in Section 27.2).
- Intermittent connection: in a real setting the link will drop. The design must assume that it may drop at any moment and be able to send the missing data and resynchronize when it recovers.
- The communication topology: can every robot communicate with every other, or only with its neighbors? This bears directly on the choice of architecture (Section 27.3).

Three: Data Synchronization

- Time synchronization: this is the difficulty most easily underrated in a multi-robot system. Every robot's clock differs a little, and without calibration the fusion will be misaligned in time. Chapters 5 and 10 stressed the importance of time synchronization repeatedly, and in a multi-machine system its effect is greater still. The usual practices are the network time protocol or timing from GNSS (Section 5.1 noted that GNSS supplies precise timing as well).
- Coordinate synchronization: the problem of unifying coordinate frames mentioned in Section 27.1. Besides estimating it by map fusion, a common starting point or a known marker in the environment may be used (the practice of Section 27.5).
- Consistency of version: when the map is updated (after a loop closure correction, say), every robot needs the newest version. If one robot acts on an old map, conflict may follow.

Four: Security

- Communication security: if the communication between robots is eavesdropped on or tampered with, an attacker may inject wrong map data and mislead the whole team. Encryption and authentication are necessary.
- Data privacy: the images an inspection robot takes may contain faces and private spaces. In settings such as a campus or a hospital the collection, storage, and access rights of the data must be regulated explicitly. This is not a technical question only but one of ethics and of law.
- Physical safety: with several robots working at once the risk to the people around them rises. The system needs an emergency stop for all of them, and every robot should have an independent safety floor (so that it does not run out of control even having lost communication).
- Fault isolation: when one robot fails or behaves abnormally, the system should be able to isolate it so that its wrong data do not contaminate the global map—which is exactly the extension of Section 27.2's outlier rejection to the level of the system.

The Close of Part VII

Part VII set out from the milliwatt-level micro platform of Chapter 25, passed through the drone navigation of Chapter 26, and arrives at the group collaboration of this chapter, presenting one clear line of thought: when the ability of a single robot is constrained, the answer is not to make it bigger and stronger but to make it smaller and more frugal and then to have more of them work together.

This line of thought agrees with nature's own strategy—an ant or a bee can do little by itself, yet the colony accomplishes astonishing things. The technology of this part is

exactly the foundation on which a team of robots acquires such collective intelligence.

In the next part we return to the most practical level: how all of this is actually made to run. Part VIII, ROS / Python / C++ Implementation, will take students through setting up a development environment, implementing 2D LiDAR mapping and navigation, running visual SLAM, and completing a full project. The theory has come this far; it is time to make the robot really move.

27.8 Three Practical Boundaries of Multi-Robot SLAM

This chapter has measured three things, and each points to a practical boundary. This section gathers them together.

One: the Boundary on Number, Decided by the Site and Not by the Budget

Figure 27-1 showed the speedup flattening at about twelve machines. That number is not universal—it is decided together by the size of the site, the complexity of its topology, and the sensing radius. In planning a fleet, students should first run the simulation of Figure 27-1 on their own site, find the saturation point that belongs to them, and only then decide how many to buy.

A rough way to estimate it is to divide the total path length of the site by the distance one machine can walk within its endurance, which gives how many are needed at least; and the saturation point usually falls between one and two times that number. Machines added beyond the saturation point have their value in fault tolerance and in the division of labor, not in speed.

Two: the Boundary on Trust—Better to Fuse Too Little Than to Fuse Wrongly

The numbers Figure 27-2 gives are extreme: one wrong edge makes the system twice

as bad as no fusion at all. This asymmetry decides a design principle for multi-robot systems—the threshold for fusion should be set stricter than for a single-machine loop closure.

Concrete practices include requiring more inliers before a cross-robot match is accepted, requiring the same match to be detected over several consecutive frames, and running a check of geometric consistency before acceptance (that the transformations among three robots should compose end to end back to the origin, say). All these checks make the system miss some correct loop closures, but by the figures of Figure 27-2 that price is far smaller than accepting one wrong edge.

Three: the Boundary on Bandwidth—Order First, Then Compress

Faced with a limit on bandwidth, the intuitive practice is to compress the data—lower the image resolution, reduce the number of feature points, sparsify the key frames. Figure 27-3 points to another dimension often overlooked: selection. Sending the same volume of data, choosing the right things to send can improve the result two or three times over.

The practical order is to select first (decide which to send) and compress after (decide how finely each is sent). Reverse the order and you will spend the bandwidth evenly across all the candidates, with the result that none is sent clearly enough and none can be used.

	What this chapter measured	What it means for design
Number of machines	Twelve flatten at a 7.4-fold speedup	The saturation point is decided by the site;

		simulate before buying
Sharing the map	With eight, it shortens the time by 82 percent	Map fusion is the premise of a multi-machine system, not a bonus
A wrong loop closure	One takes the ATE from 0.118 to 0.507 meters	The threshold for fusion must be stricter than for a single-machine loop closure
The robust kernel	It stopped every one, but with no wrong edges the accuracy fell from 0.118 to 0.319	Insurance has its premium; two-stage handling obtains both
Ordering the candidates	10 ordered ≒ 30 at random	Order first, then compress

Table 27-3 What the Three Experiments of This Chapter Imply for Design

Formulas and Procedures

One: the Coordinate Transformation Across Robots

Let the world frames of robots A and B be W_A and W_B. The heart of map fusion is to find the relative transformation between them:

$$T_{W_A \leftarrow W_B}$$

Once this transformation is had, any pose of B can be converted into A's frame:

$$p_{W_A} = T_{W_A \leftarrow W_B} \cdot p_{W_B}$$

This is exactly Chapter 10's composing transformations is multiplying matrices, applied to several robots. If the team has N robots, N − 1 relative transformations

suffice in principle to bring every map into one frame (forming a spanning tree, which answers to the idea of Section 22.6).

Two: the Objective Function of a Multi-Robot Pose Graph

Once the pose graphs of all the robots are merged, the objective of the overall optimization is:

$$\min \Sigma_{\text{edges within a robot}} \|e\|^2_\Sigma + \Sigma_{\text{edges across robots}} \rho(\|e\|^2_\Sigma)$$

The first term holds each robot's own odometry and loop closure constraints, the second the loop closure constraints across robots.

Note particularly the robust kernel ρ outside the second term—the risk of error in a cross-robot edge is far higher than in an edge within one robot (Section 27.2), and a robust kernel must therefore protect it. This detail embodies the idea of treating things differently according to their credibility, of a piece with the Kalman gain of Chapter 11 and the information matrix of Chapter 12.

Three: Estimating the Volume of Communication

The communication a multi-robot system requires may be roughly estimated as:

$$\text{total communication} \approx N \times (N-1) \times \text{the data exchanged between each pair} \times \text{the rate}$$

Note the growth with the square of N—which is exactly the mathematical reason the centralized and fully connected architectures are hard to scale. With 10 robots the number of pairs is 45; raise it to 20 and the number of pairs leaps to 190.

This formula explains the necessity of Section 27.2's prioritizing of loop closure candidates and Section 27.3's decentralized architecture: as N grows, the actual

communication must somehow be held far below this upper bound—communicating only when needed, and sending only what is most valuable.

Four: The Complete Procedure of Multi-Robot SLAM

Figure 27-5 The Processing Flow of Collaborative Multi-Robot SLAM

【Onboard each robot】

each runs single-machine SLAM independently (Chapters 13–25) → its own pose graph and map

↓ extract a condensed representation (key frame descriptors, sparse features)

【Communication】

exchange candidate information by priority (bandwidth is limited; send only what is valuable)

↓

【Cross-robot loop closure detection】

place recognition matching → geometric verification → outlier rejection

↓ obtain the relative transformation across robots

【Map fusion】

establish the cross-robot edges → merge the pose graphs

↓

【Global optimization】

pose graph optimization (with a robust kernel) → a globally consistent map

↓

【Distribution】send the corrected results back to every robot

Cases

Case 27-1 The Subterranean Challenge: the Ultimate Examination Hall of Multi-Robot SLAM

Case 1-2 in Chapter 1 introduced the DARPA Subterranean Challenge; here that competition is looked at again from the multi-robot point of view.

The competition required a team of robots to explore, map, and find target objects autonomously in man-made tunnels, an underground city, and natural caves. That setting gathers very nearly the worst version of every challenge in this chapter:

- Communication is terrible: radio attenuates severely underground, and a robot often loses contact entirely once it has gone deep—which is exactly the extreme case of Section 27.3's sporadic, intermittent communication.
- The environment is degenerate: the geometry of a tunnel repeats itself highly (the structural degeneracy of Section 17.3), the risk of perceptual aliasing is very high, and the probability of a wrong loop closure rises greatly.
- The team is heterogeneous: the competing teams generally combined wheeled robots, quadruped robot dogs, and drones, each suited to different terrain (the complementary capabilities of Section 27.1).

This competition gave rise directly to several important systems. LAMP and LAMP 2.0 were developed precisely to handle underground multi-robot SLAM in the DARPA Subterranean Challenge; LAMP 2.0 introduced a centralized prioritization mechanism based on a graph neural network, predicting the outcome of every cross-robot loop closure candidate after pose graph optimization.

This practice of using AI to predict which loop closure candidates are worth handling is most suggestive: when resources (communication, computation) are limited, rather than treat every candidate alike, predict the value by a learned method

and then allocate the resources. It is the same thought as the prioritizing of Section 27.2, by a different technical means.

The lesson for students is that an extreme setting is often the hotbed of a technical breakthrough. The systems entered in the Subterranean Challenge have since become important foundations of multi-robot SLAM—a hard problem forced out good answers.

Case 27-2 How Badly Does the System Break When Communication Is Unreliable

Papers on multi-robot SLAM mostly evaluate under good communication. But in a real setting communication is often broken and restored by turns—how much does this affect the system?

Research has quantified this question specifically, and the result is quite striking: against a baseline of reliable communication, the error of Swarm-SLAM rises 5.08 times and that of Kimera-Multi 6.17 times.

In other words, the same system may see its error worsen five or six times when communication is unreliable. The figure reminds us that handsome laboratory data need not be reproducible in the field.

It also explains why recent research sets more and more store by the conditions of communication. Section 27.4's treating communication quality as a planning resource is exactly such a response: rather than accept the consequences of poor communication passively, plan the route actively so as to stay connected, or design an architecture that degrades gracefully when the link drops.

For students doing a project this is an important reminder: in evaluating your multi-robot system, be sure to include a test under poor communication. Testing only under ideal conditions will overestimate the real ability of the system severely. This is of a piece with the spirit of Section 14.6's common failure cases and Section 26.5's challenge test—seeking out the boundary of failure actively is responsible engineering practice.

Case 27-3 From Simulation to the Field: Development Tools for a Heterogeneous Team

The development of a multi-robot system has one practical difficulty: the cost of hardware. The price of ten robot dogs is a heavy burden for most laboratories, and the labor and the space needed to run an experiment with ten robots are considerable too.

Simulation is therefore a necessary tool in multi-robot research. Open-source simulation environments designed for heterogeneous multi-robot work have appeared in recent years. There is, for instance, an open-source simulator and data collection pipeline built on an Unreal Engine 5 simulator, which resolves key architectural limitations of earlier frameworks so as to support unmanned aerial and ground vehicles operating together in large-scale, photorealistic, dynamic environments; it introduces a new waypoint-following ground vehicle controller matching the existing drone control interface, and supplies a shared navigation stack for heterogeneous platforms covering mapping, traversability analysis, planning, and control.

Tools of this kind are of particular value in teaching: students can build teams of ten or twenty robots in simulation and test task allocation, map fusion, and

communication strategies without a corresponding quantity of physical hardware. Once the algorithm has been shown to work in simulation, two or three real robots suffice to verify the crucial parts.

This is exactly the path of learning this book has recommended since Chapter 8: simulate first, then cheap hardware, and pursue scale afterwards. A multi-robot system magnifies the value of that principle to the utmost—without simulation, multi-robot research can hardly move a step.

Implementation Activities

Implementation 27-1 Simulating Collaborative Mapping by Several Robots

Purpose: to experience in full, through simulation, the central questions of multi-robot SLAM—map fusion, the limits of communication, task allocation, and the risk of a wrong loop closure. This activity is mainly a simulation and needs no fleet of real robots.

Part One: Fusing the Maps of Two Robots (everyone)

1. Reuse the simulated environment of Implementation 25-1 in Chapter 25 (or the 2D mapping simulation of Chapter 13), create two virtual robots setting out from different corners of the room, and have each explore by its own route and build its own map.
2. Observe the problem: display the two maps laid one on the other and you will find that they do not agree at all—because each takes its own starting point as its origin (the problem of unifying coordinate frames in Section 27.1).

3. Fuse by hand: supposing you know the relative position of the two starting points, transform coordinates by Formula One, bring B's map into A's frame, and display them together. Observe whether the fused map is complete.
4. Estimate the transformation automatically: let the two robots' routes overlap in part, register the two maps (or the points in the overlapping region) with the ICP of Chapter 17, and estimate the relative transformation automatically. Compare the error against the truth.
5. Add drift: give both robots' pose estimates an accumulated error, fuse again, observe how the map is thrown out of register, and correct it by pose graph optimization (Chapter 12).

Part Two: An Experiment on the Limits of Communication (everyone, or undergraduate)

1. Set a communication range: rule that the two robots may exchange data only when they are closer than some distance. Simulate their exploring separately and meeting now and then.
2. Compare three conditions of communication: connected at all times, communicating only on meeting, and dropping at random (no communication with probability 30 percent at every step, say), and compare the quality of the final map.
3. Quantify the effect: compute the map error under the three conditions, set it against the five- to sixfold increase in error mentioned in Case 27-2, and see whether your simulation shows a similar trend.
4. Count the data: record the total communication under the three conditions and think about the trade-off between the volume of communication and the quality of the map.

Part Three: Task Allocation and Wrong Loop Closures (undergraduate)

1. Compare the efficiency of exploration: let 1, 2, and 4 robots each explore the same room until it is wholly covered, and record the number of steps each needs. Draw the curve of the number of robots against the time to completion and see whether the benefit grows linearly (hint: usually it does not, because some regions are explored twice).
2. Strategies of task allocation: compare the efficiency of random exploration against division by area, setting this against the discussion of Section 27.5.
3. An experiment on a wrong loop closure: add a wrong cross-robot loop closure edge by hand (connecting some pose of A wrongly to an unrelated pose of B, say), run the pose graph optimization, and observe how the map is spoiled.
4. Make it robust: add a robust kernel to the optimization (scipy's soft_l1 and similar loss functions will serve), run it again, and observe whether the damage of the wrong loop closure is suppressed. This verifies directly the work of ρ in Formula Two.

Discussion Questions

1. What do the two maps look like laid one on the other with no coordinate transformation? What new problem of multi-robot SLAM against a single machine does this show?
2. How large is the error of the relative transformation ICP estimates automatically? What happens if the two robots' routes hardly overlap at all?
3. By what factor does the map error rise when the condition of communication changes from connected at all times to communicating only on meeting? How does this compare with the data of Case 27-2?

4. Going from 1 robot to 4, by what fraction is the time to completion shortened? Why is it not a quarter?
5. (Undergraduate) After the wrong loop closure was added, which parts of the map were spoiled worst? By how much did the robust kernel suppress the damage? What does this imply for the design of a real system?

Extended Challenge (university level)

First: implement a simple prioritizing of loop closures—when communication is limited and only some candidates can be sent, design a scoring mechanism (by how far apart the two stretches of trajectory a loop closure connects are, say, or whether the robots involved already have other links), send the high-scoring candidates first, and compare the result with random selection. This corresponds to the central idea of Section 27.2. Second: simulate a heterogeneous team—add one robot that is fast but has a small sensing range and one that is slow but has a large one, design a task allocation strategy that plays to each one's strength, and compare the efficiency with a homogeneous team. This answers to the complementary capabilities of Section 27.1 and the ground-air collaboration of Section 26.7.

Further Reading

1. Y. Tian et al., "Kimera-Multi: Robust, Distributed, Dense Metric-Semantic SLAM for Multi-Robot Systems," IEEE T-RO, 2022. (The representative work in distributed multi-robot semantic SLAM; the code is public.)
2. P.-Y. Lajoie and G. Beltrame, "Swarm-SLAM: Sparse Decentralized Collaborative SLAM Framework for Multi-Robot Systems," IEEE RA-L, 2024. (A decentralized ROS 2 framework, with the loop closure prioritization technique.)

3. Y. Chang et al., "LAMP 2.0: A Robust Multi-Robot SLAM System for Operation in Challenging Large-Scale Underground Environments," IEEE RA-L, 2022. (The representative system of the DARPA Subterranean Challenge.)
4. P. Schmuck and M. Chli, "CCM-SLAM: Robust and Efficient Centralized Collaborative Monocular SLAM," Journal of Field Robotics, 2019. (The classic of centralized collaborative SLAM.)
5. P.-Y. Lajoie et al., "DOOR-SLAM: Distributed, Online, and Outlier Resilient SLAM for Robotic Teams," IEEE RA-L, 2020. (The distributed architecture and outlier rejection.)
6. The awesome-Collaborative-SLAM open-source list (GitHub). (A complete collection of papers and code on collaborative SLAM, and a good place to begin research.)
7. S. Macenski et al., "Robot Operating System 2: Design, Architecture, and Uses in the Wild," Science Robotics, 2022. (ROS 2's support for several robots, the foundation of the implementation in Part VIII.)

Summary

1. The four advantages of multi-robot SLAM are speed, fault tolerance, complementary viewpoints, and complementary capabilities; its three great new challenges are unifying the coordinate frames, the widening of data association, and the limits of communication. Its core technologies are the single-machine technologies (SLAM, place recognition, pose graph optimization, outlier rejection) extended over several coordinate frames and several sources of data.

2. Map fusion has three steps: detect the encounter, estimate the relative transformation, and fuse and optimize; the greatest risk is a wrong cross-robot loop closure—the space of candidates is larger, perceptual aliasing is worse, and the consequences are more serious, so robust outlier rejection is an indispensable component, while prioritizing the loop closure candidates obtains the greatest benefit within a limited bandwidth.
3. The three architectures each have their trade-offs: the centralized is simple to fuse but has a communication bottleneck and a single point of failure and cannot scale to a large team; the distributed and decentralized perform peer-to-peer pose graph optimization, scale well, and are resilient to a break in communication, at the price of complexity in synchronization and a burden of onboard computation; and a decentralized solution depending only on sporadic, intermittent communication suits a large setting better.
4. The onboard, edge, and cloud tiers divide the labor by timeliness, by volume of data, and by privacy; the edge-assisted practice maintains autonomy with onboard visual odometry and offloads the heavy computation to a server; and recent work treats communication quality as a planning resource, making the navigation cliff a manageable event of degradation rather than a hard limit.
5. Campus inspection by several robot dogs adopts the three-tier onboard, edge, and cloud architecture; the task may be allocated by area, dynamically by frontier, or by capability; the practical questions include aligning the initial coordinates (which a common charging station or an AprilTag simplifies), communication coverage, giving way to one another, and a battery rotation.
6. An AI agent plays in a multi-machine system the planner of tasks, the reader of anomalies, and the arbiter of coordination; sharing semantics and not only

geometry keeps the team's understanding of the environment consistent, but three limitations—latency, hallucination, and dependence on communication—must be kept in mind.

7. The four engineering questions: a formation must balance collision avoidance, the communication chain, and the efficiency of coverage; communication must face limited bandwidth and intermittent connection; data synchronization covers consistency of time, of coordinates, and of version; and security covers encrypted communication, data privacy, physical safety, and fault isolation.

Carrying Forward

This chapter has quantified three key parameters of bringing more of them. The speedup of collaborative exploration flattens at about twelve machines at 7.4 times, and the difference made by sharing a map is very large—with eight machines, sharing shortens the time to completion by 82 percent, and without it the speedup stops dead at about 1.9 however many are added. The positive value of map fusion is to bring the ATE from 0.369 meters down to 0.118, but let a single wrong cross-robot edge in and it worsens to 0.507 meters, which is worse than no fusion at all, while a robust kernel holds it steady at 0.319. As for bandwidth, ordering by algebraic connectivity needs only ten cross-robot loop closures to achieve what thirty picked at random achieve.

Part VII is complete. From the milliwatt-level micro platform of Chapter 25, through the drone navigation of Chapter 26, to the group collaboration of this chapter, one clear line of thought has emerged: when the ability of a single robot is constrained, the answer is not to make it bigger and stronger but to make it smaller and more frugal and then to have more of them work together—which agrees with the strategy of the

ant and the bee.

In the next part we return to the most practical level: how all of this is actually made to run. Part VIII, ROS / Python / C++ Implementation, will take students through setting up a development environment, implementing 2D LiDAR mapping and navigation, running visual SLAM, and completing a full project with its assessment. The theory has come this far; it is time to make the robot really move.